CG3201: Machine Learning and Deep Learning

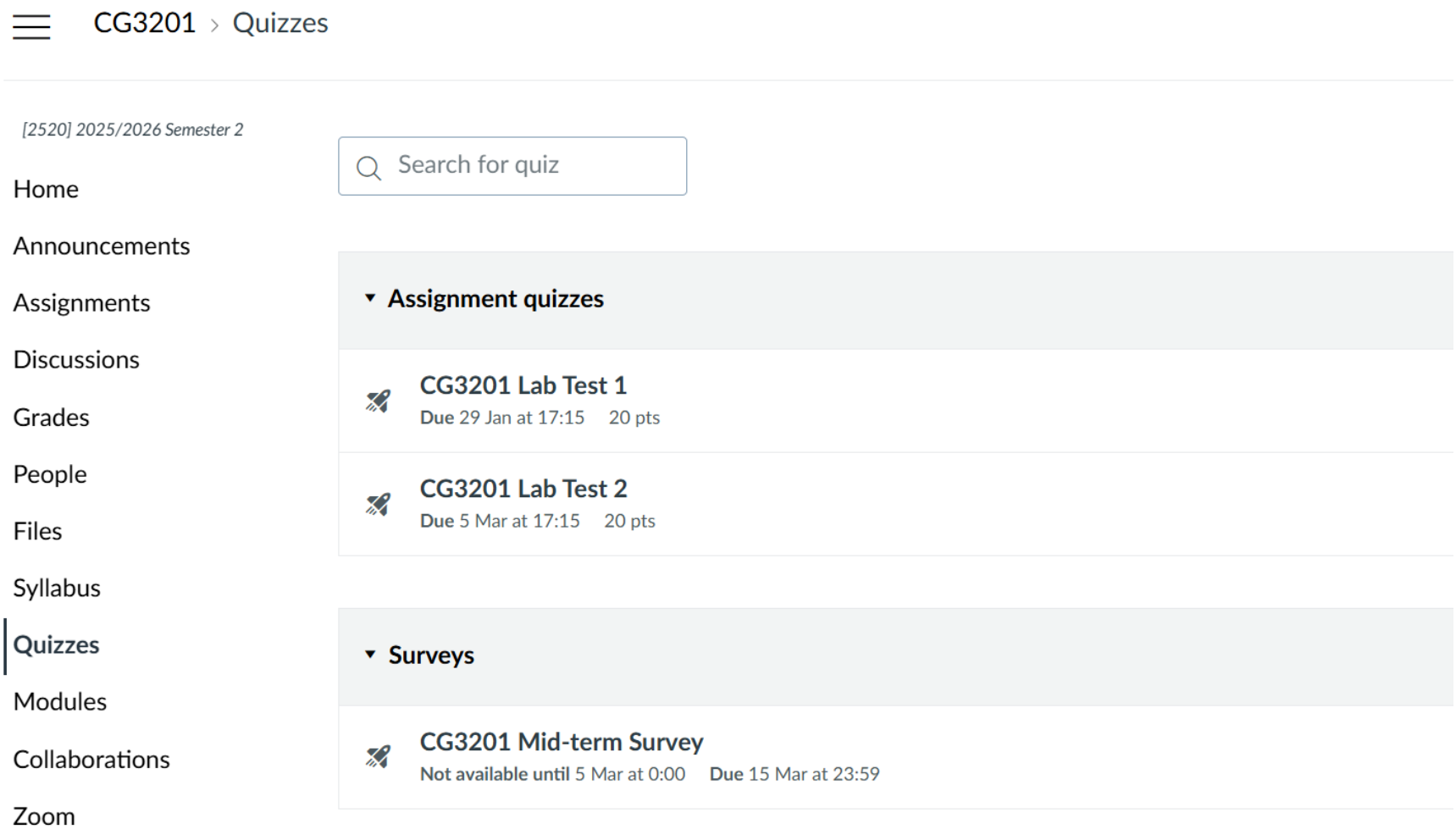
Lecture 5: Convolution Neural Network

Dr. WANG Si
si.wang@nus.edu.sg

Electrical and Computer Engineering Department
National University of Singapore

Mid-Course Feedback Survey

Your Feedback Matters — Help Us Improve



https://canvas.nus.edu.sg/courses/85106/assignments/241675

OVERVIEW OF PART II (Si's Part)

- **Tentative Schedule**

  1. CNN & CNN Architectures – Weeks 7&8
  2. RNNs & Transformers – Weeks 9&10
  3. Generative Models– Weeks 11&12

- **(Credit to)**

  Justin Johnson, "Deep Learning for Computer Vision"
  https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/schedule.html

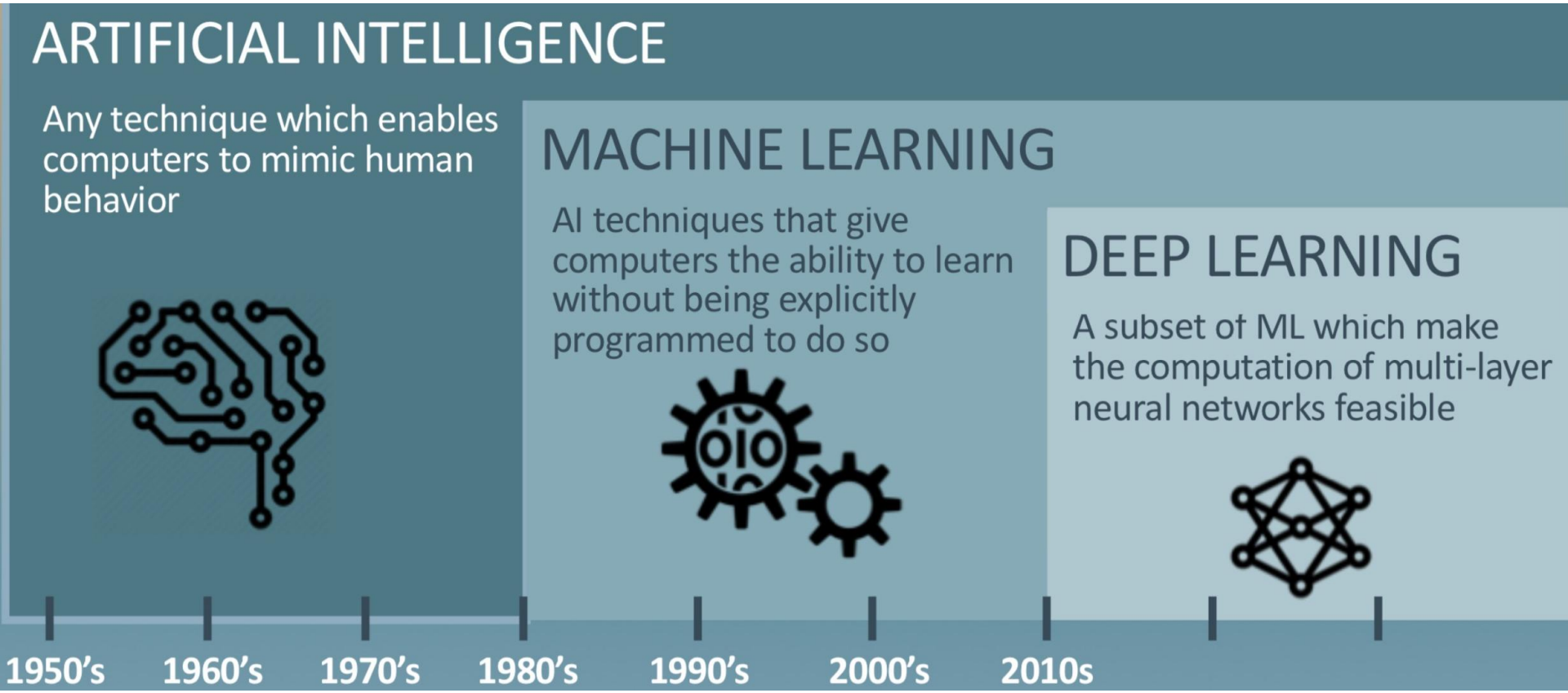
Lecture 5's AGENDA

- We will discuss:
  - Recap of Multilayer Perceptron (MLP)
  - Key Components in Convolution Neural Network (CNN)

At the end of this lecture, you should be able to:

- ✓ Understand Backpropagation
- ✓ Explain the Key Components in CNNs.

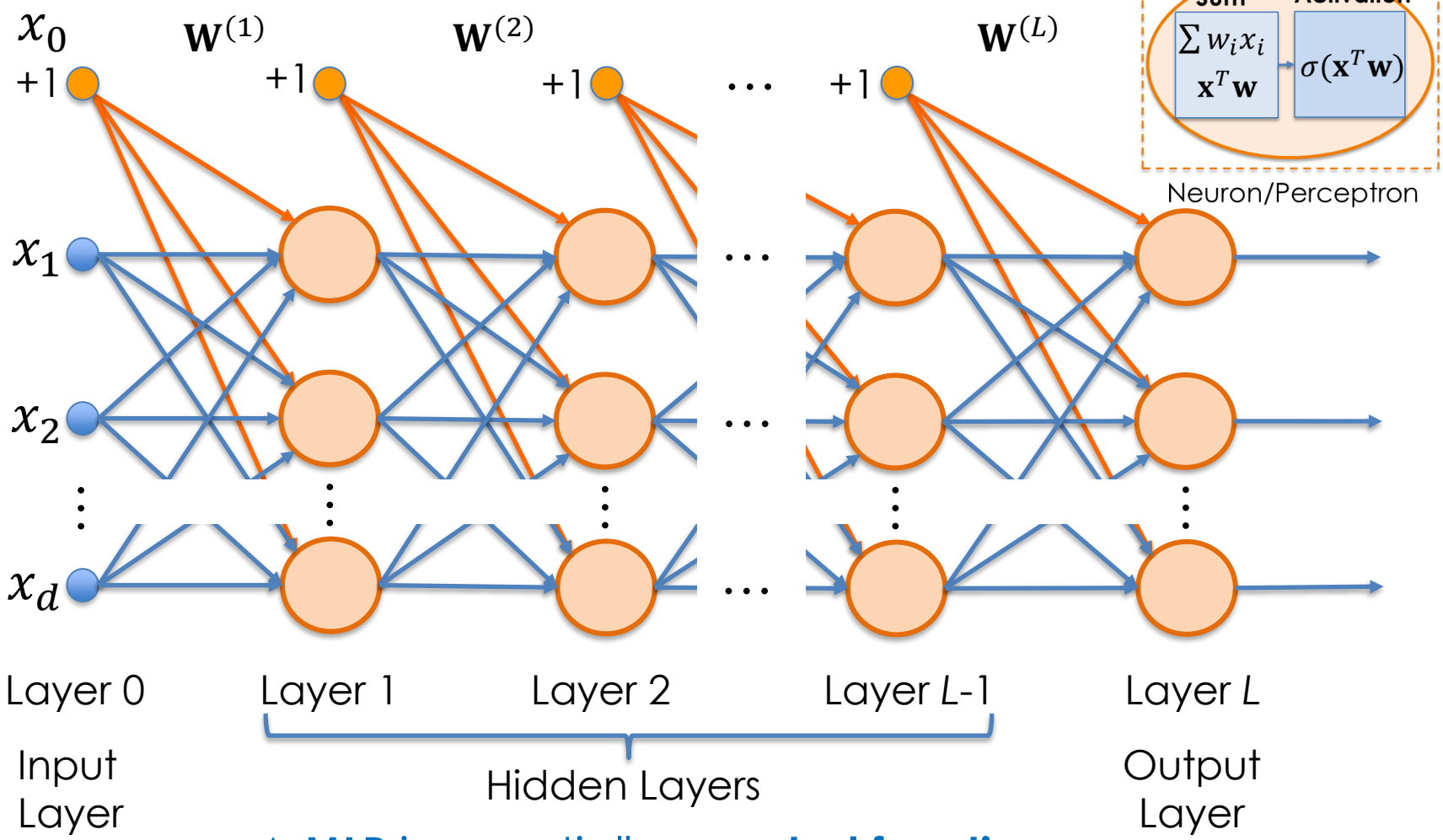
Recap: AI & ML & DL



Recap: Multilayer Perceptron (MLP)

Recap: Multilayer Perceptron (MLP)

➢ Architecture



A **MLP** is essentially a **nested function**

$$\mathbf{f}_{\mathbf{W}}(\mathbf{x}) = \boldsymbol{\sigma}^{(L)}([1, \dots \boldsymbol{\sigma}^{(2)}([1, \boldsymbol{\sigma}^{(1)}(\mathbf{x}^T\mathbf{W}^{(1)})]\mathbf{W}^{(2)}) \dots]\mathbf{W}^{(L)})$$

Recap: Multilayer Perceptron (MLP)

➢ Architecture

Example: A MLP of 2 layers has been constructed as:

$$\mathbf{f}_{\mathbf{W}}(\mathbf{x}) = \boldsymbol{\sigma}\left(\left[1, \boldsymbol{\sigma}\left(\mathbf{x}^T \mathbf{W}^{(1)}\right)\right] \mathbf{W}^{(2)}\right),$$

where $\mathbf{x} = [1,1,3]^T$, $\mathbf{W}^{(1)} = \mathbf{W}^{(2)} = \begin{bmatrix} -1 & 0 \\ 0 & -1 \\ 1 & 0 \end{bmatrix}$, $\sigma(z) = \max(0, \mathrm{z})$.

Compute network output $\mathbf{f}_{\mathbf{W}}(\mathbf{x})$.

Recap: Multilayer Perceptron (MLP)

➢ Architecture

Example: A MLP of 2 layers has been constructed as:

$$\mathbf{f}_{\mathbf{W}}(\mathbf{x}) = \boldsymbol{\sigma}\left(\left[1, \boldsymbol{\sigma}\left(\mathbf{x}^T \mathbf{W}^{(1)}\right)\right] \mathbf{W}^{(2)}\right),$$

where $\mathbf{x} = [1,1,3]^T$, $\mathbf{W}^{(1)} = \mathbf{W}^{(2)} = \begin{bmatrix} -1 & 0 \\ 0 & -1 \\ 1 & 0 \end{bmatrix}$, $\sigma(z) = \max(0, z)$.

Compute network output $\mathbf{f}_{\mathbf{W}}(\mathbf{x})$.

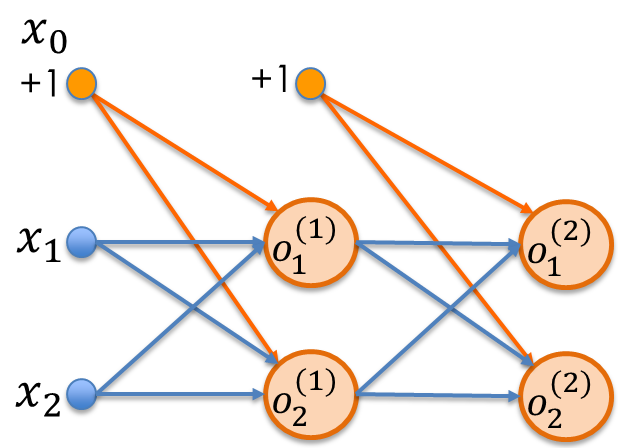
Solution:

$$\boldsymbol{\sigma}\left(\mathbf{x}^T \mathbf{W}^{(1)}\right) = \boldsymbol{\sigma}\left([1,1,3] \begin{bmatrix} -1 & 0 \\ 0 & -1 \\ 1 & 0 \end{bmatrix}\right)$$

$$= \boldsymbol{\sigma}([2, -1]) = [2,0]$$

$$\boldsymbol{\sigma}\left(\left[1, \boldsymbol{\sigma}\left(\mathbf{x}^T \mathbf{W}^{(1)}\right)\right] \mathbf{W}^{(2)}\right) = \boldsymbol{\sigma}\left([1,2,0] \begin{bmatrix} -1 & 0 \\ 0 & -1 \\ 1 & 0 \end{bmatrix}\right)$$

$$= \boldsymbol{\sigma}([-1, -2]) = [0,0]$$



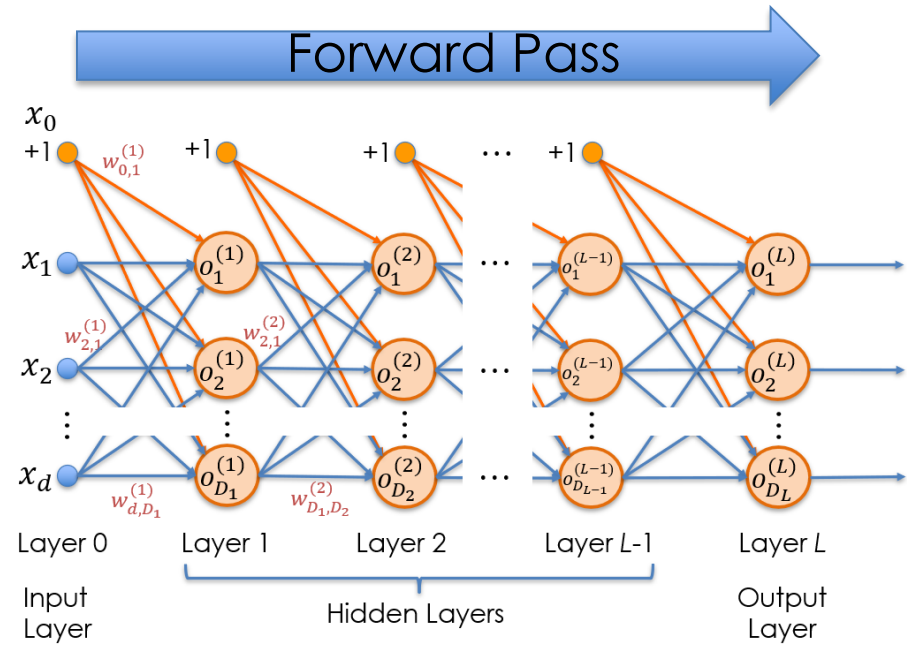
Recap: Multilayer Perceptron (MLP)

➢ Learning (W)

1. Random Initialization of **W**
2. Forward Pass (fixed **W**)
   i. Compute output responses
   ii. Compute errors at each output
3. **Backward Pass** (update **W**)

   i.e.,**Backpropagation**

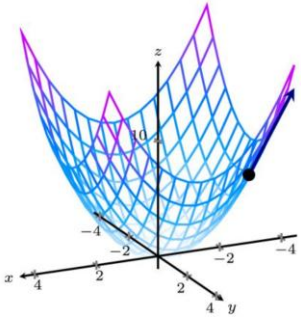
   i. Propagate the errors backward
   ii. Update **W**



Backward Pass

$$\mathbf{W}_{new} = \mathbf{W}_{old} - \eta \boxed{\nabla_{\mathbf{W}} J(\mathbf{W}_{old})}$$

The direction and rate of fastest increase



In essence, Backpropagation= Gradient Decent!

Recap: Multilayer Perceptron (MLP)

➢ Learning (W)

$$\nabla_{\mathbf{W}} J(\mathbf{W}_{old}) = \sum \nabla_{\mathbf{W}} \mathcal{L}(\mathbf{W}_{old})$$

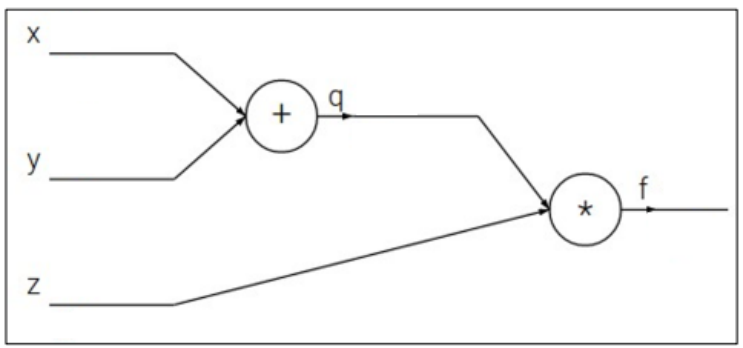
Problem: Derive $\nabla_{\mathbf{W}} \mathcal{L}$ on paper is very tedious

Change loss?
Change activation function?

Solution: Computational Graphs

e.g., $f(x, y, z) = (x + y) \times z$

where x=-2, y=5, z=-4



Step 1: Forward pass (compute outputs)

q=x+y=3, f=q×z=-12

Step 2: Backward pass (compute derivatives)

Goal: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

i. Compute derivative of each node output with respect to each corresponding input.

$* : \frac{\partial f}{\partial z}, \frac{\partial f}{\partial q}$ $\quad + : \frac{\partial q}{\partial x}, \frac{\partial q}{\partial y}$

ii. Chain rule

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q}\frac{\partial q}{\partial x}, \frac{\partial f}{\partial y} = \frac{\partial f}{\partial q}\frac{\partial q}{\partial y}, \frac{\partial f}{\partial z}$$

Recap: Multilayer Perceptron (MLP)

➢ Learning (W)

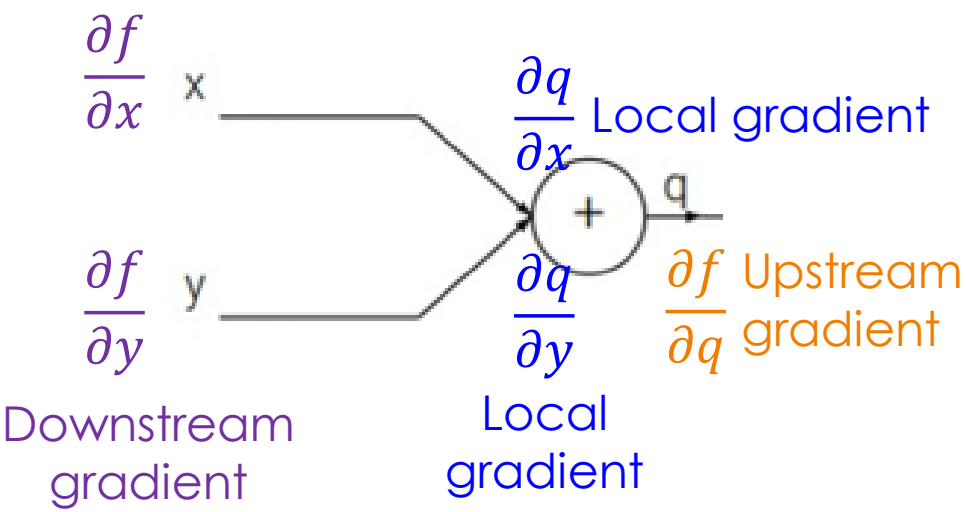
$$\nabla_{\mathbf{W}} J(\mathbf{W}_{old}) = \sum \nabla_{\mathbf{W}} \mathcal{L}(\mathbf{W}_{old})$$

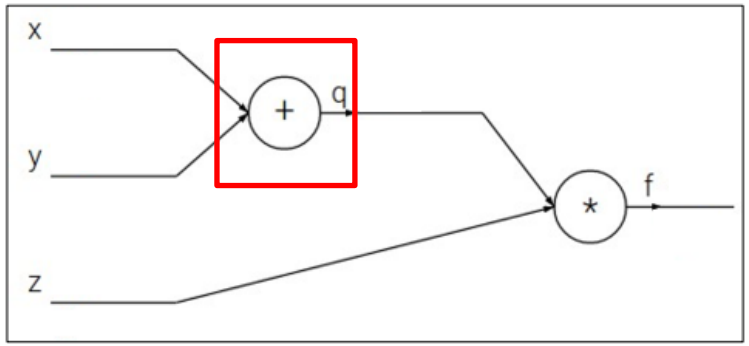
Problem: Derive $\nabla_{\mathbf{W}} \mathcal{L}$ on paper is very tedious

Change loss?
Change activation function?

Solution: Computational Graphs

e.g., $f(x, y, z) = (x + y) \times z$
where x=-2, y=5, z=-4



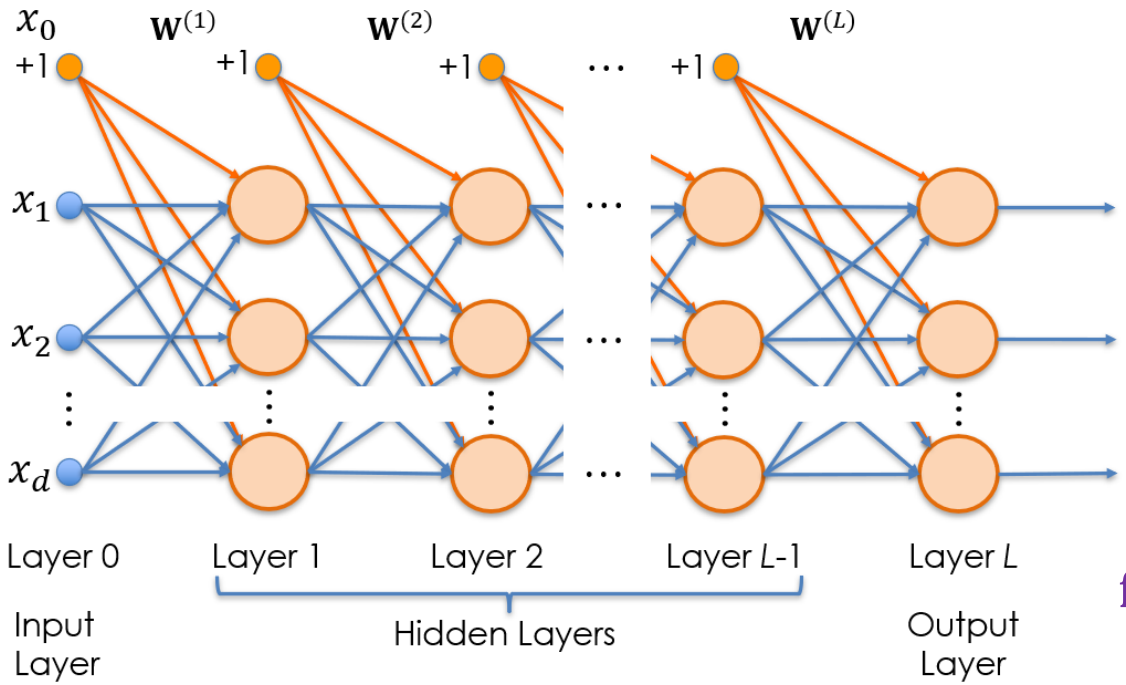


Downstream gradient = Upstream gradient x Local gradient

Recap: Multilayer Perceptron (MLP)

➢ Prediction (Forward Pass)

$$\mathbf{x}_{new} = \begin{bmatrix} 0.2 \\ 0.3 \\ 0.4 \end{bmatrix}$$



$$\mathbf{f}_{\mathbf{W}}(\mathbf{x}_{new}) = \begin{bmatrix} 0.2 \\ 0.1 \\ 0.7 \end{bmatrix}$$

Forward Pass

Fixed **W**, Compute $\mathbf{f}_{\mathbf{W}}(\mathbf{x}_{new})$

Recap: Multilayer Perceptron (MLP)

➢ Image Classification

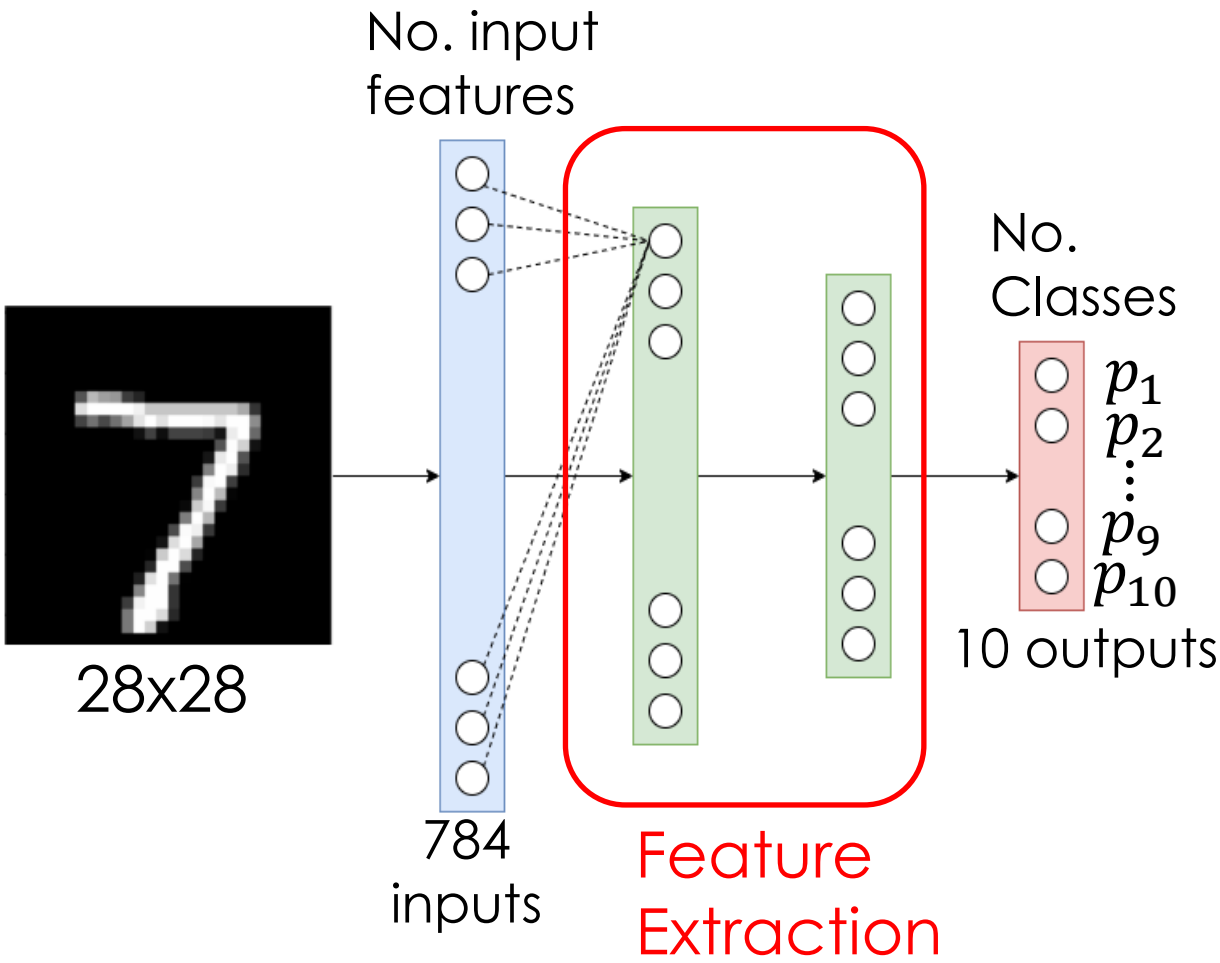
Image is stored as a matrix in a computer

0	2	15	0	0	11	10	0	0	0	0	9	9	0	0	0
0	0	0	4	60	157	236	255	255	177	95	61	32	0	0	29
0	10	16	119	238	255	244	245	243	250	249	255	222	103	10	0
0	14	170	255	255	244	254	255	253	245	255	249	253	251	124	1
2	98	255	228	255	251	254	211	141	116	122	215	251	238	255	49
13	217	243	255	155	33	226	52	2	0	10	13	232	255	255	36
16	229	252	254	49	12	0	0	7	7	0	70	237	252	235	62
6	141	245	255	212	25	11	9	3	0	115	236	243	255	137	0
0	87	252	250	248	215	60	0	1	121	252	255	248	144	6	0
0	13	113	255	255	245	255	182	181	248	252	242	208	36	0	19
1	0	5	117	251	255	241	255	247	255	241	162	17	0	7	0
0	0	0	4	58	251	255	246	254	253	255	120	11	0	1	0
0	0	4	97	255	255	255	248	252	255	244	255	182	10	0	4
0	22	206	252	246	251	241	100	24	113	255	245	255	194	9	0
0	111	255	242	255	158	24	0	0	6	39	255	232	230	56	0
0	218	251	250	137	7	11	0	0	0	2	62	255	250	125	3
0	173	255	255	101	9	20	0	13	3	13	182	251	245	61	0
0	107	251	241	255	230	98	55	19	118	217	248	253	255	52	4
0	18	146	250	255	247	255	255	255	249	255	240	255	129	0	5
0	0	23	113	215	255	250	248	255	255	248	248	118	14	12	0
0	0	6	1	0	52	153	233	255	252	147	37	0	0	4	1
0	0	5	5	0	0	0	0	0	14	1	0	6	6	0	0

0	2	15	0	0	11	10	0	0	0	0	9	9	0	0	0
0	0	0	4	60	157	236	255	255	177	95	61	32	0	0	29
0	10	16	119	238	255	244	245	243	250	249	255	222	103	10	0
0	14	170	255	255	244	254	255	253	245	255	249	253	251	124	1
2	98	255	228	255	251	254	211	141	116	122	215	251	238	255	49
13	217	243	255	155	33	226	52	2	0	10	13	232	255	255	36
16	229	252	254	49	12	0	0	7	7	0	70	237	252	235	62
6	141	245	255	212	25	11	9	3	0	115	236	243	255	137	0
0	87	252	250	248	215	60	0	1	121	252	255	248	144	6	0
0	13	113	255	255	245	255	182	181	248	252	242	208	36	0	19
1	0	5	117	251	255	241	255	247	255	241	162	17	0	7	0
0	0	0	4	58	251	255	246	254	253	255	120	11	0	1	0
0	0	4	97	255	255	255	248	252	255	244	255	182	10	0	4
0	22	206	252	246	251	241	100	24	113	255	245	255	194	9	0
0	111	255	242	255	158	24	0	0	6	39	255	232	230	56	0
0	218	251	250	137	7	11	0	0	0	2	62	255	250	125	3
0	173	255	255	101	9	20	0	13	3	13	182	251	245	61	0
0	107	251	241	255	230	98	55	19	118	217	248	253	255	52	4
0	18	146	250	255	247	255	255	255	249	255	240	255	129	0	5
0	0	23	113	215	255	250	248	255	255	248	248	118	14	12	0
0	0	6	1	0	52	153	233	255	252	147	37	0	0	4	1
0	0	5	5	0	0	0	0	0	14	1	0	6	6	0	0

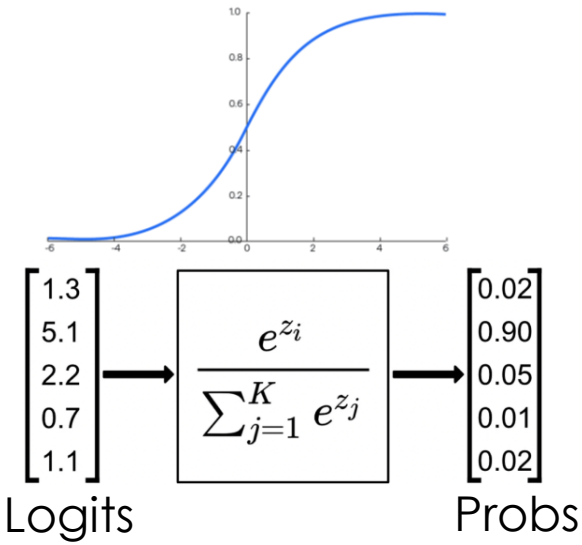
Recap: Multilayer Perceptron (MLP)

➢ Image Classification



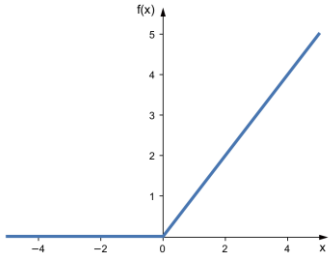
If output layer (l =L):

$$\sigma = Softmax$$



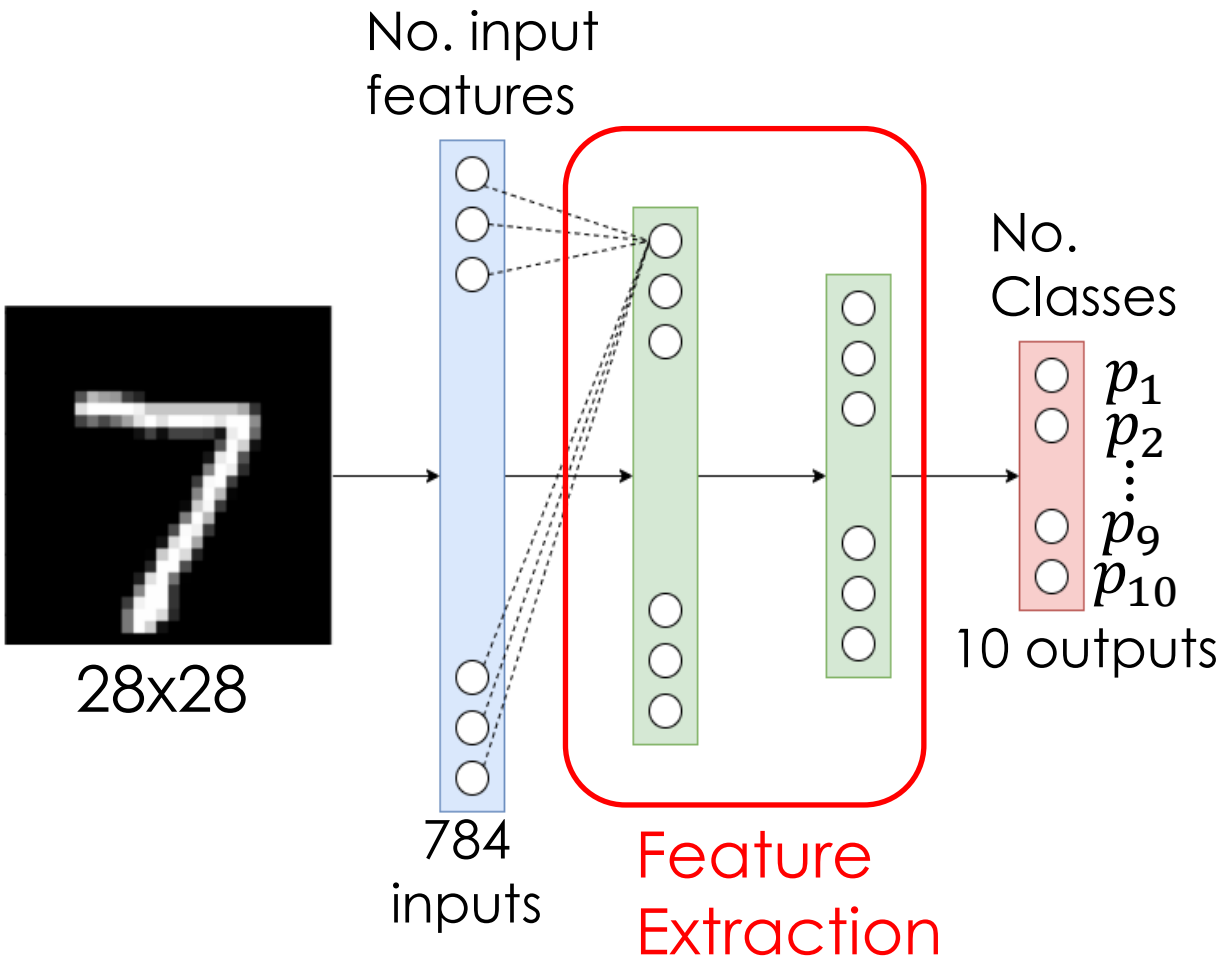
If hidden layer (l <L):

$$\sigma = ReLU$$



Recap: Multilayer Perceptron (MLP)

➢ Image Classification



Loss Function:

$$\mathcal{L}(\mathbf{W}) = -\sum_{k=1}^{K} y_k \ln(p_k)$$

Categorical cross-entropy loss

e.g., An image $\mathbf{x}$ from class c,

$$\mathbf{y} = [0, \dots, 0, 1, 0, \dots 0]$$

$$y_1 \qquad y_c \qquad y_K$$

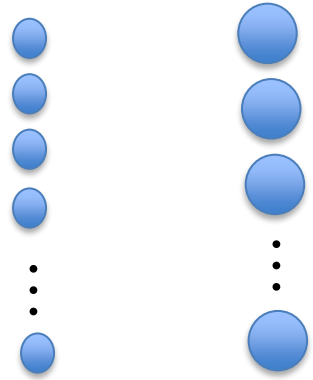
$$\mathcal{L}(\mathbf{W}) = -\ln(p_c)$$

Only the target/ground-truth class contributes

Key Limitations of MLP

- **Key Limitations in Image Classification**

200x200



10K neurons in the first layer

How many input features?

40000x10000≈400 millions parameters for the first layer

Parameter explosion

Flattening

Loss of Spatial Structure

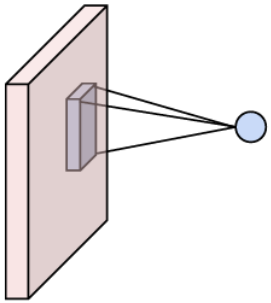
Convolution Neural Network

(CNN)

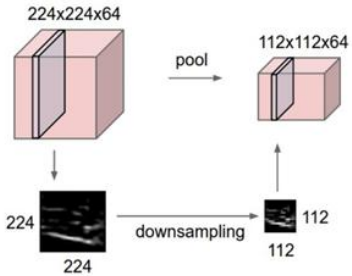
CNNs

➢ Components of A Typical CNN

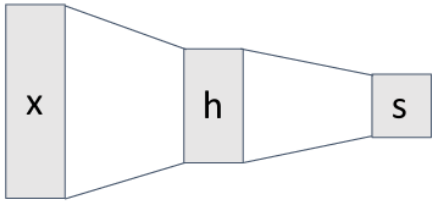
Convolution Layers



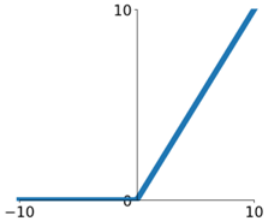
Pooling Layers



Fully-Connected Layers



Activation Function



Normalization

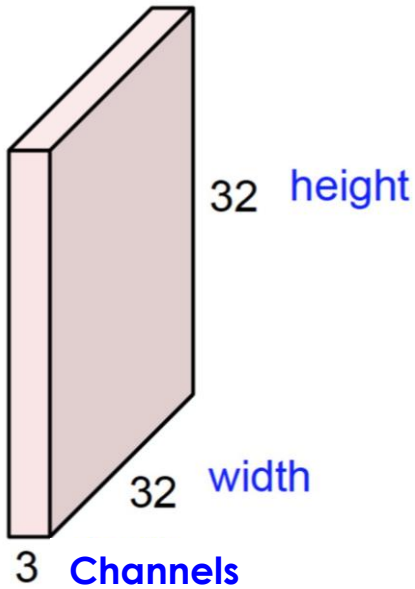
$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

CNNs-Convolution Layer

➢ Convolution

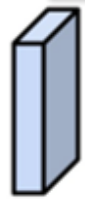
Input: 3x32x32 image -> Preserve the spatial structure

Filters always extend the full depth of the input volume

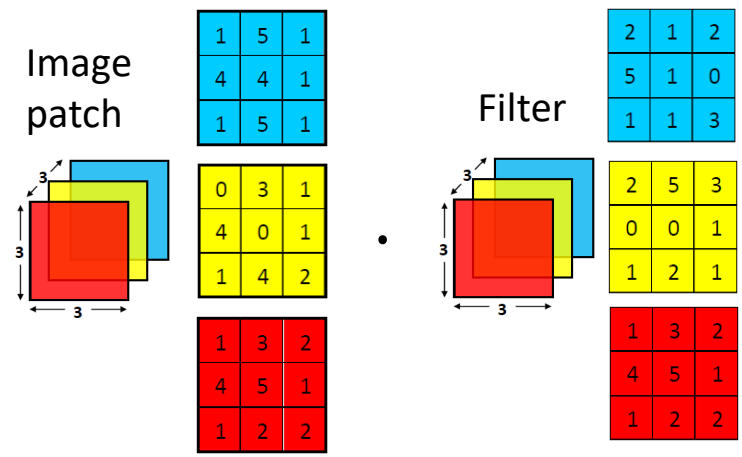


3x5x5 filter (Weight Matrix)

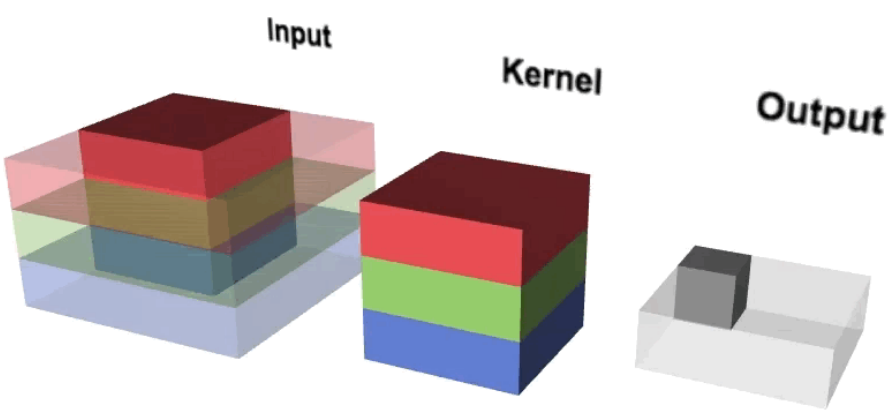
Kernel size



e.g.,



$= 137$



Convolve the filter with the image. i.e., "slide over the image spatially, computing dot products"

CNNs-Convolution Layer

➢ Convolution

Example: Suppose the input size is 2x3x3 and the filter size is 2x2x2, what is the output after convolving the filter with the input (slide over each spatial location)?

Input channel 1:

$$\begin{bmatrix} 1 & -1 & 1 \\ 1 & -1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$$

Filter channel 1:

$$\begin{bmatrix} 1 & -1 \\ 1 & -1 \end{bmatrix}$$

Input channel 2:

$$\begin{bmatrix} 0 & 1 & 0 \\ -2 & 0 & 1 \\ -2 & 0 & 0 \end{bmatrix}$$

Filter channel 2:

$$\begin{bmatrix} -1 & 0 \\ -1 & 0 \end{bmatrix}$$

CNNs-Convolution Layer

➢ Convolution

Example: Suppose the input size is 2x3x3 and the filter size is 2x2x2, what is the output after convolving the filter with the input (slide over each spatial location)?

Input channel 1:

$$\begin{bmatrix} 1 & -1 & 1 \\ 1 & -1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$$

Filter channel 1:

$$\begin{bmatrix} 1 & -1 \\ 1 & -1 \end{bmatrix}$$

Input channel 2:

$$\begin{bmatrix} 0 & 1 & 0 \\ -2 & 0 & 1 \\ -2 & 0 & 0 \end{bmatrix}$$

Filter channel 2:

$$\begin{bmatrix} -1 & 0 \\ -1 & 0 \end{bmatrix}$$

Positive output implies?

Output: $\begin{bmatrix} 6 & -4 \\ 7 & -2 \end{bmatrix}$

Negative output implies?

Dot product: similarity measure

Convolution is fundamentally a pattern-matching operation

CNNs-Convolution Layer

➢ Convolution

Input size: **3**x32x32 image

Filter size: **3**x5x5

Output size : ?

In general

Input size: C_{in} x W x W

Filter size: C_{in} x K x K

Output size: ?

CNNs-Convolution Layer

➢ Convolution

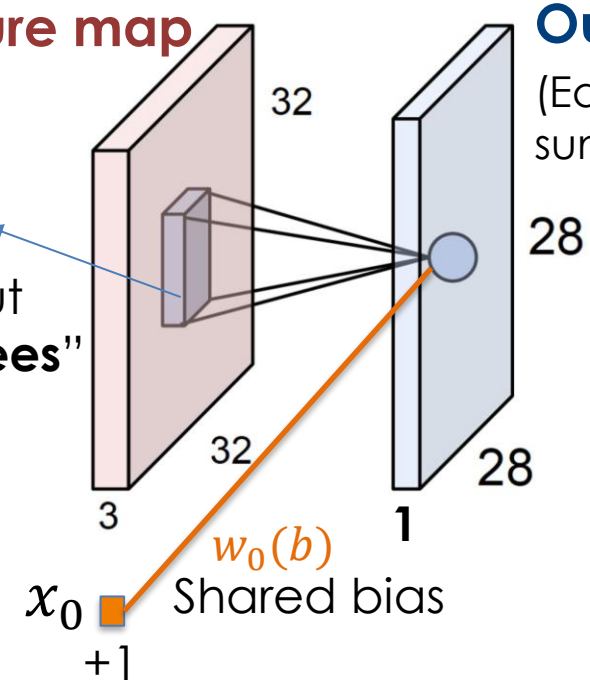
Input feature map

Output feature map

(Each entry computes weighted sum, like a neuron)

Receptive Field

The region of the **initial** input that a particular neuron "**sees**" or **is influenced by**



	MLP	CNN
input	3x32x32	~
output	1x28x28	~
No. Para	?	?

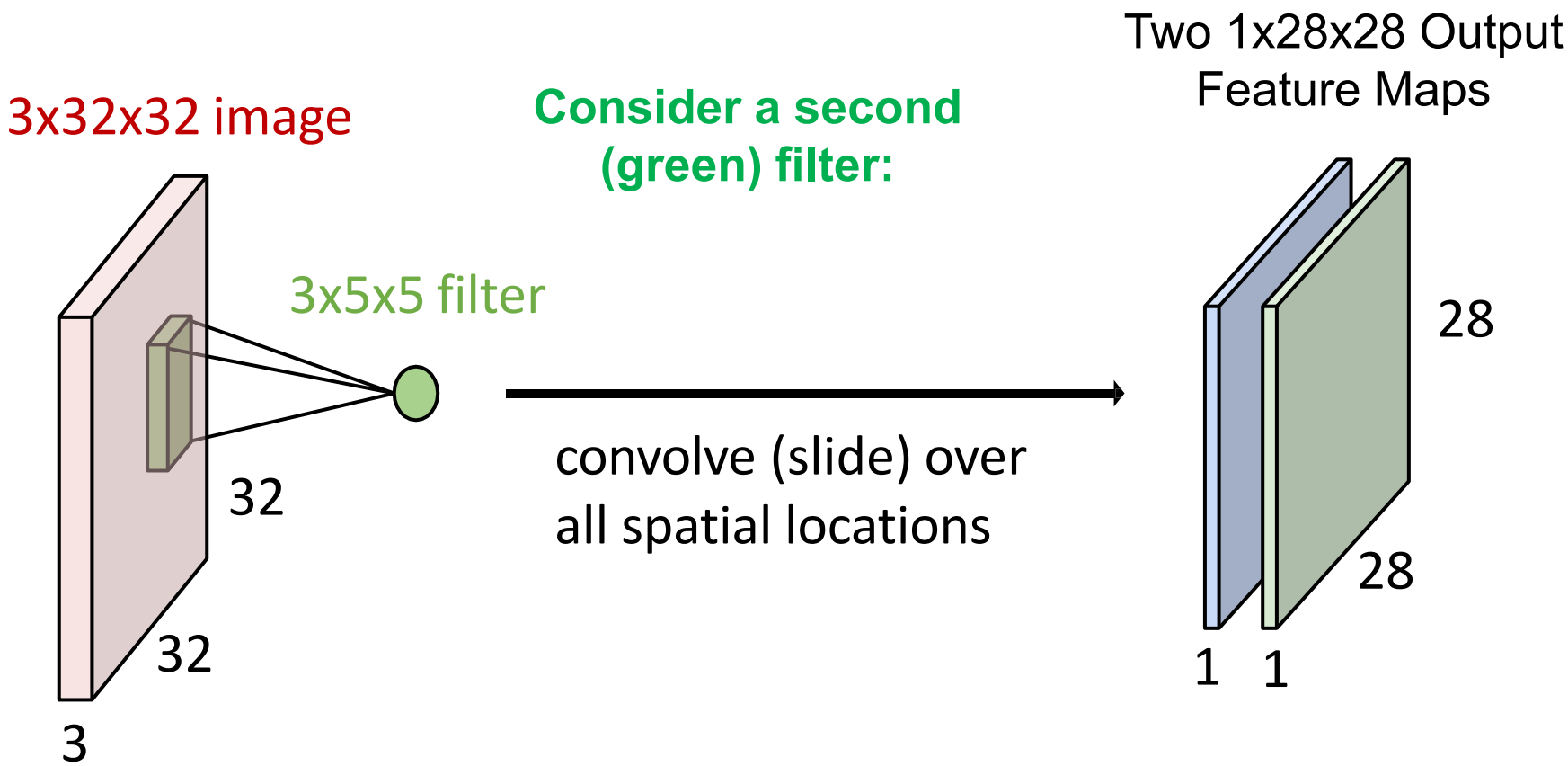
Shared Parameters: Parameter Reduction!

CNNs-Convolution Layer

➢ Convolution

Input size: **3**x32x32 image

Filter size: **3**x5x5



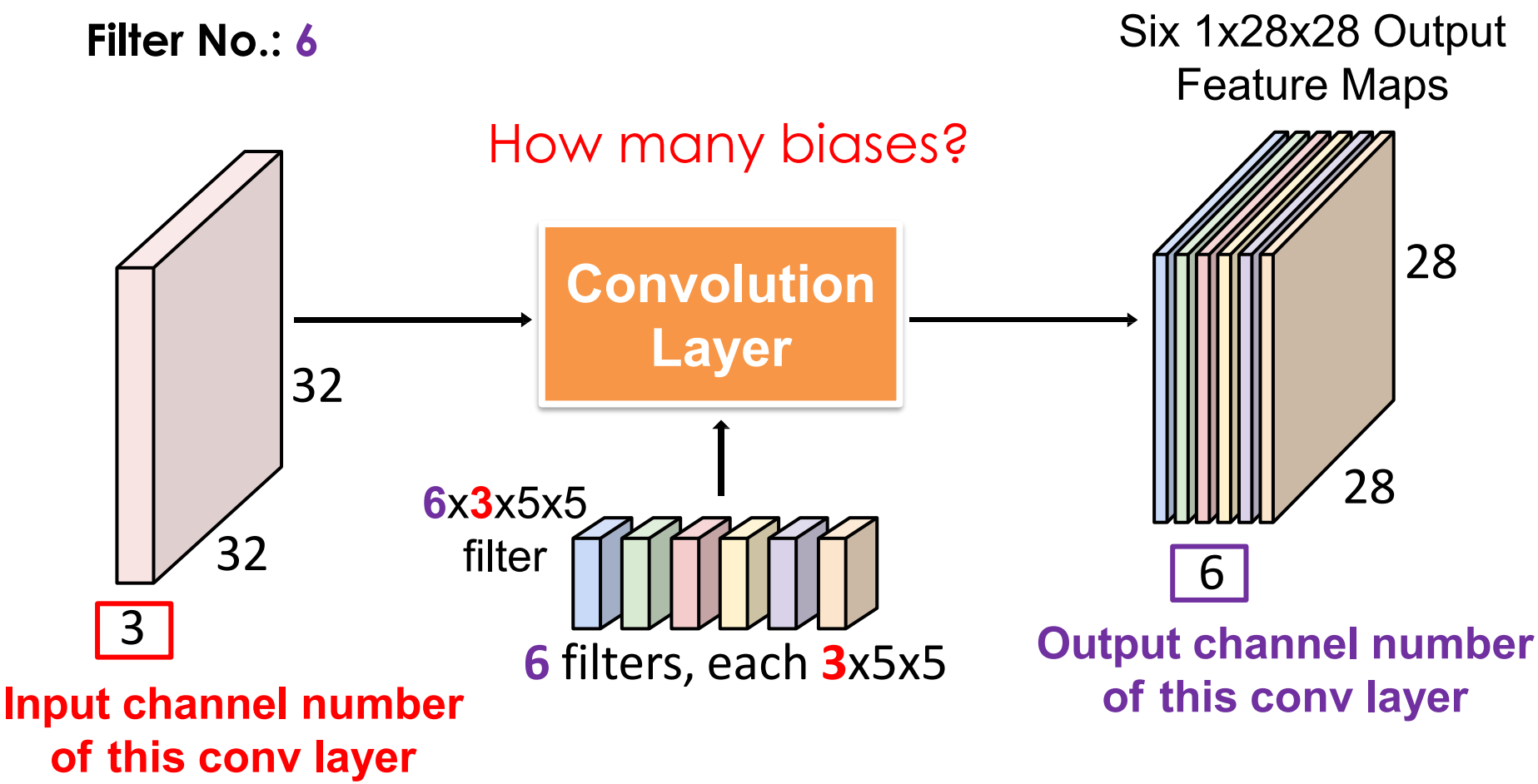
CNNs-Convolution Layer

- **Each Conv Layer is Made up of Filters and Biases**

Input size: **3**x32x32 image

Filter size: **3**x5x5

Filter No.: 6



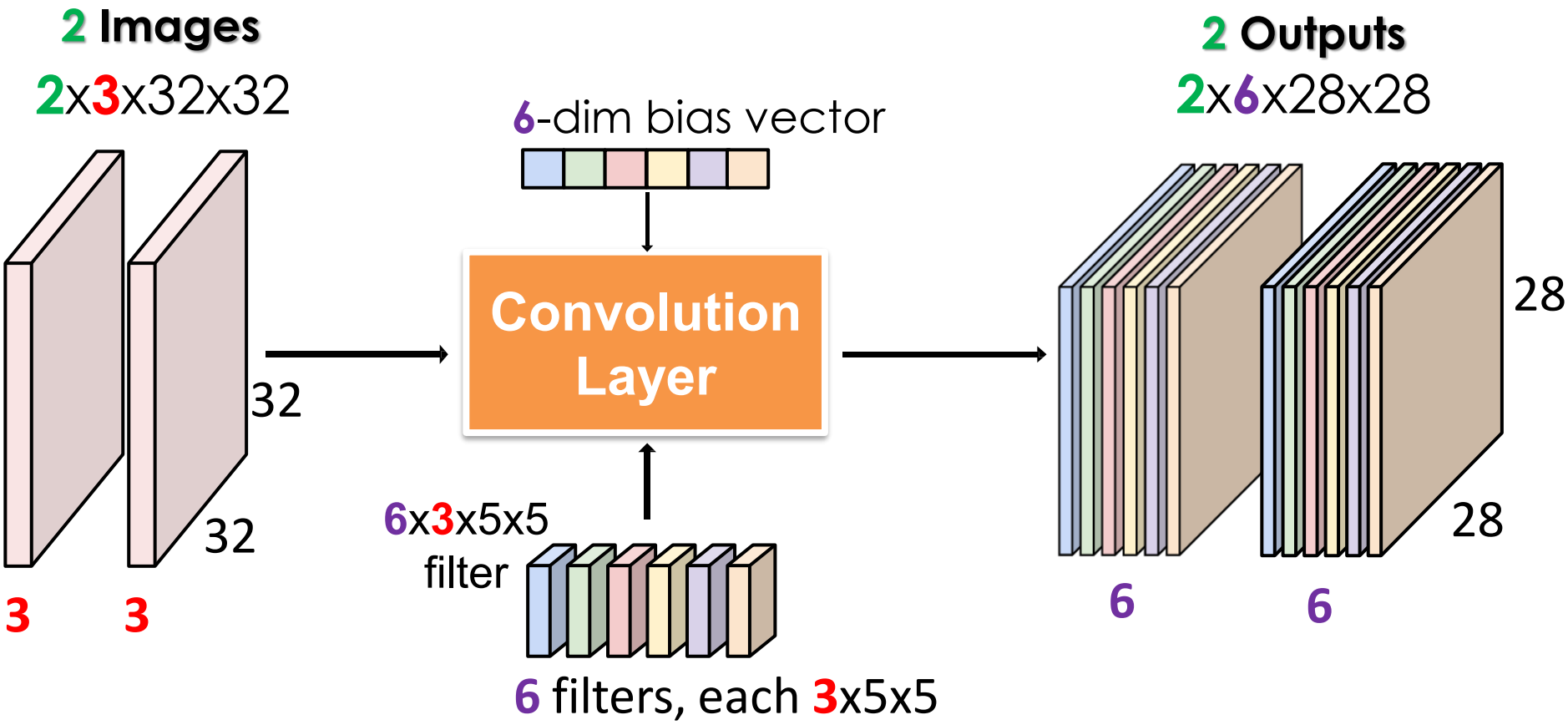
CNNs-Convolution Layer

- **Each Conv Layer is Made up of Filters and Biases**

Input size: 3x32x32 image

Filter size: 3x5x5, Filter No.: 6

Batch size: 2



CNNs-Convolution Layer

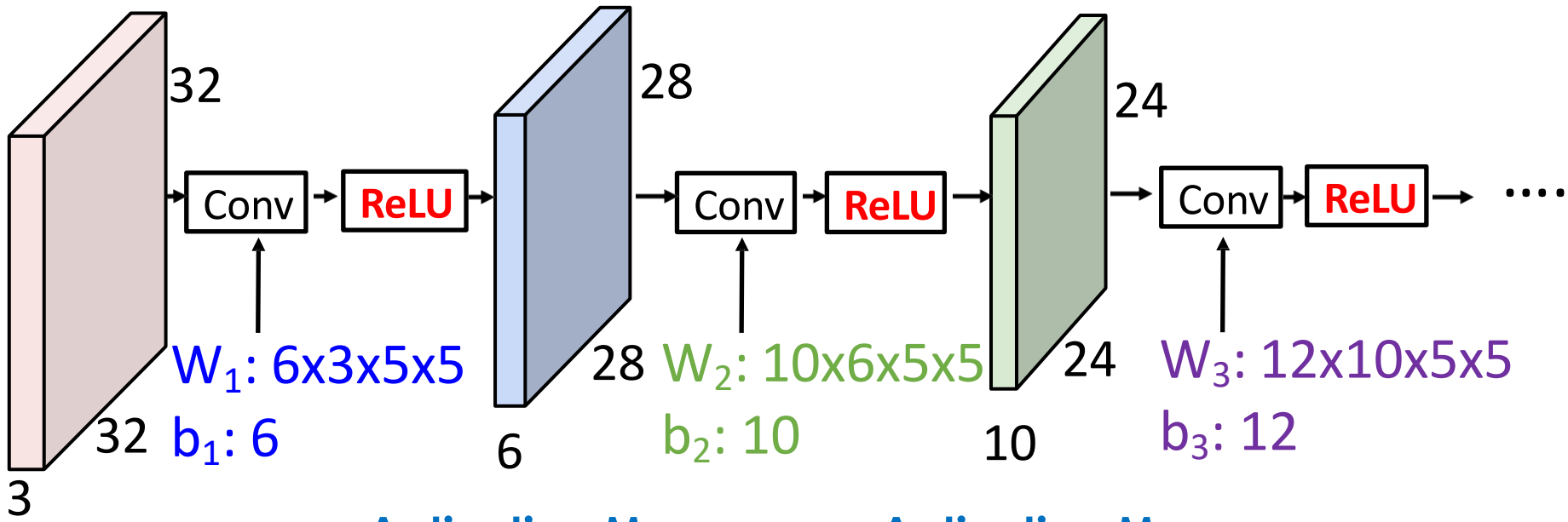
➢ Multiple Conv Layers

Problem: Activation maps "shrink" with each conv layer!

Input:
N x 3 x 32 x 32

First hidden layer:
N x 6 x 28 x 28

Second hidden layer:
N x 10 x 24 x 24



Activation Maps **Activation Maps**

CNNs-Convolution Layer

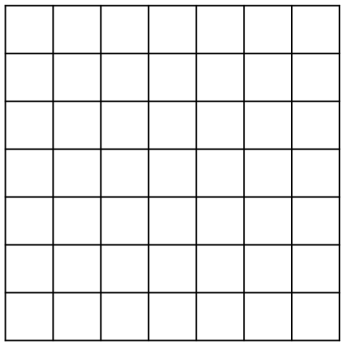
➢ Zero-Padding: Mitigate Shrinkage of Spatial Dimensions

Add zeros around the input

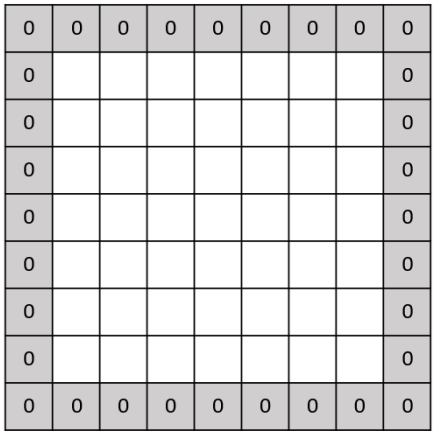
(Only show the spatial dimensions for visualization)

Assume filter size: C_{in} x 3 x 3

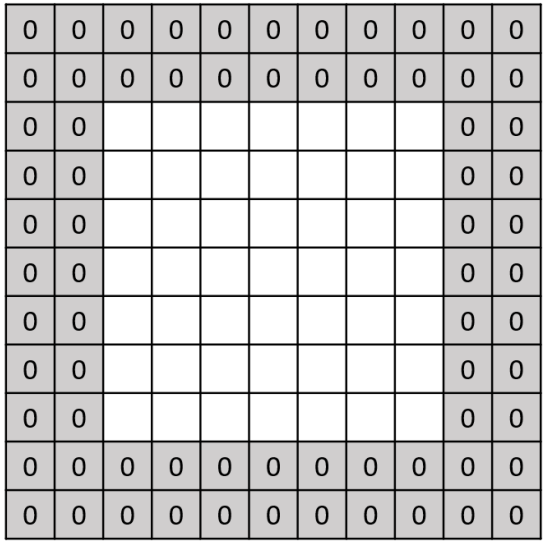
C_{in} x 7 x 7



Original Input



With the size of zero-padding P=1



With the size of zero-padding P=2

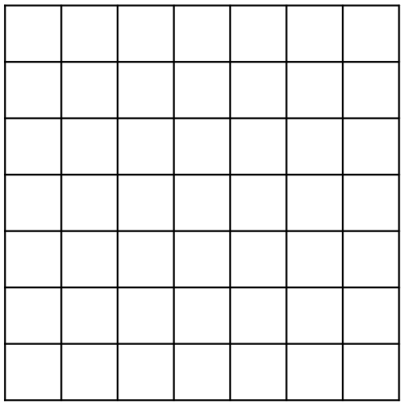
Output size (spatially):

CNNs-Convolution Layer

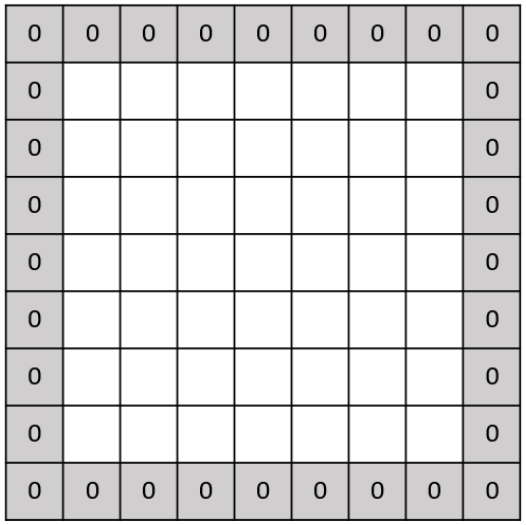
➢ Zero-Padding: Mitigate Shrinkage of Spatial Dimensions

Add zeros around the input

(Only show the spatial dimensions for visualization)



Original Input



Zero-padded Input

Input size: C_{in} x W x W

Filter size: C_{in} x K x K

Zero-padding size: P

Output size (Spatially):?

✓ **Very common Practice: <u>Same Padding</u>**

The output spatial dimension is the same as the original input spatial dimension

formula for P?

CNNs-Convolution Layer

➢ A Closer Look at Receptive Fields

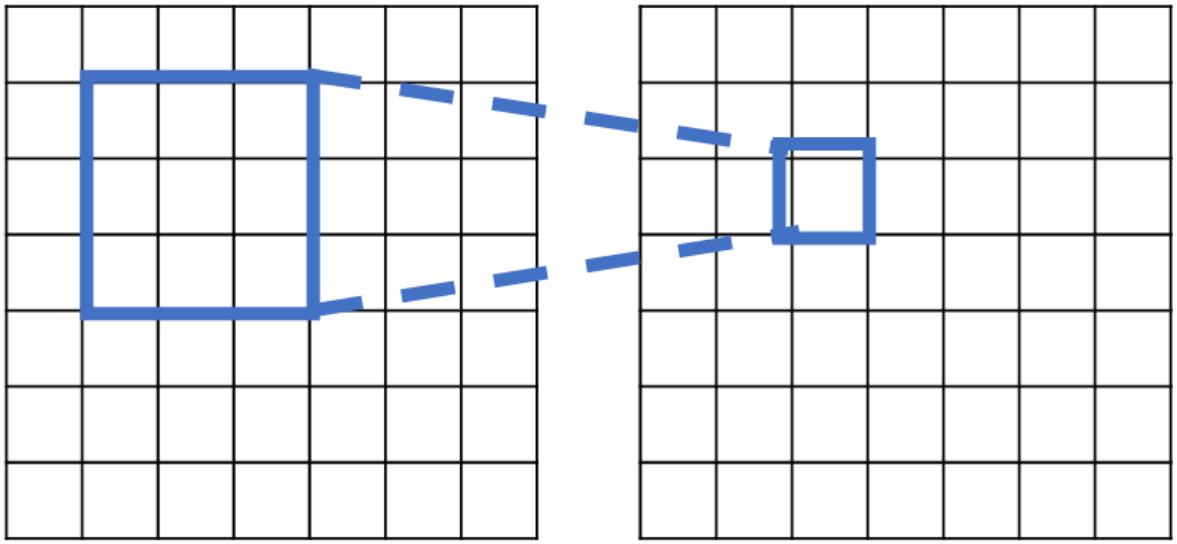
Input size: C_{in} x W x W **Kernel size: K x K**

Receptive field size (Spatially) for each output of the **first** Conv layer:

K x K

(Only show the spatial dimensions for visualization)

e.g.,
W=7,
K=3



Input Output

CNNs-Convolution Layer

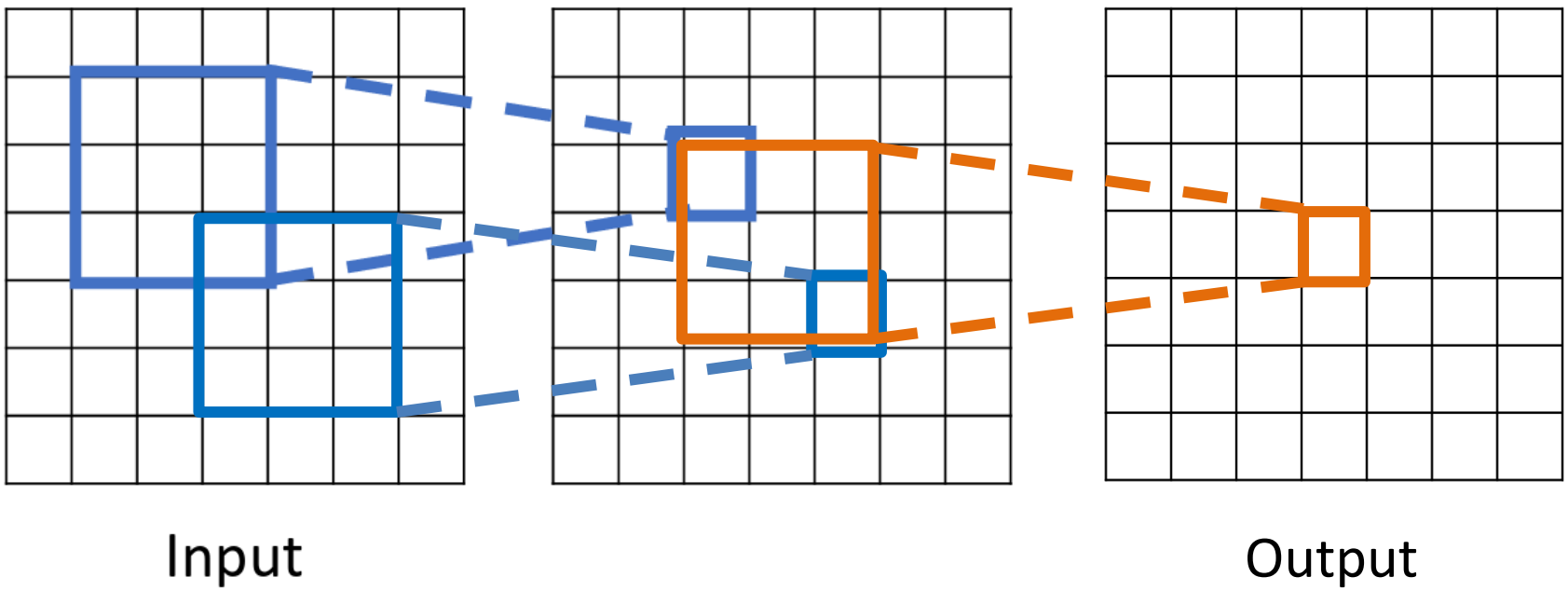
➢ A Closer Look at Receptive Fields

Input size: C_{in} x W x W **Kernel size: K x K**

Receptive field size (Spatially) for each output of the **second** Conv layer:

(2K-1) x (2K-1)

(Only show the spatial dimensions for visualization)



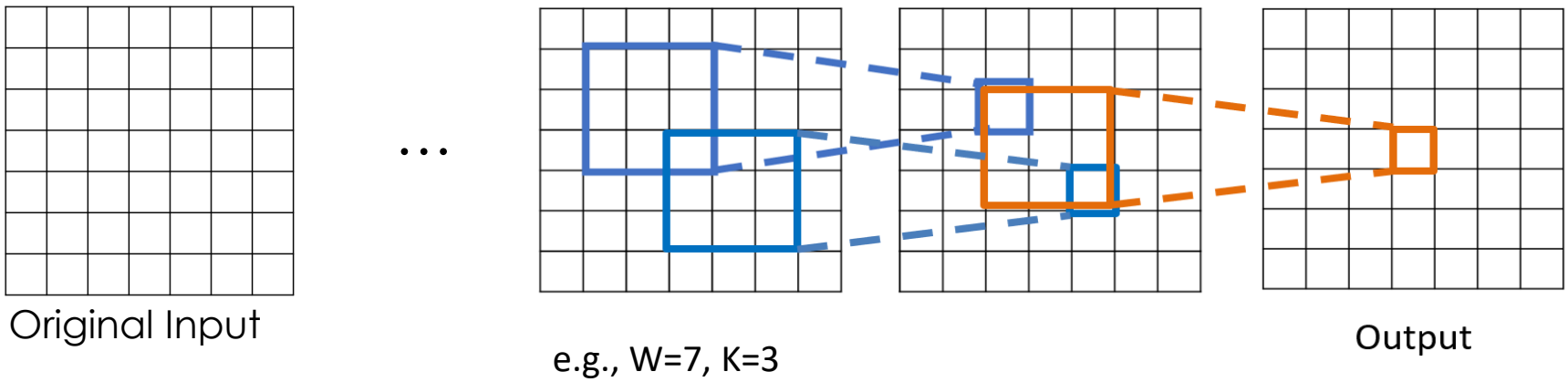
e.g., W=7, K=3

CNNs-Convolution Layer

➢ A Closer Look at Receptive Fields

Input size: C_{in} x W x W **Kernel size: K x K**

Receptive field size (Spatially) for each output of **L-th** convolution layer?

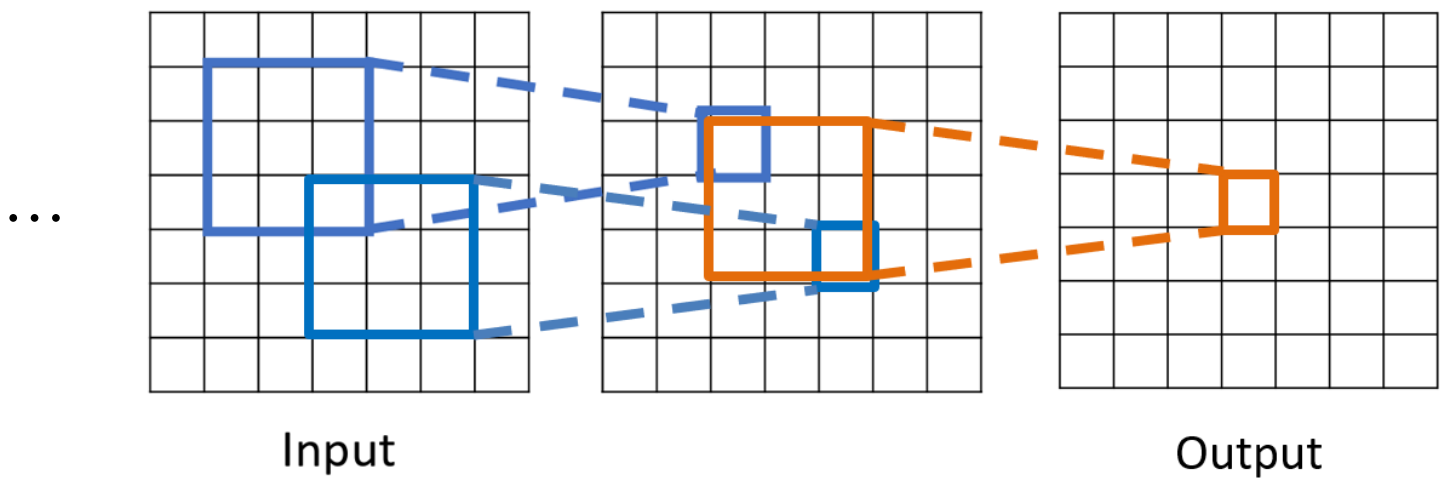


CNNs-Convolution Layer

➢ A Closer Look at Receptive Fields

Input size: C_{in} x W x W **Kernel size: K x K**

Receptive field size (Spatially) for each output of **L-th** convolution layer?



RF_3-K+1=RF_2 RF_2-K+1= RF_1 RF_1-K+1=1

RF_3= K + (K-1) + (K-1) RF_2=K + (K-1) RF_1=K

Each successive convolution adds *K*-1 to the receptive field size

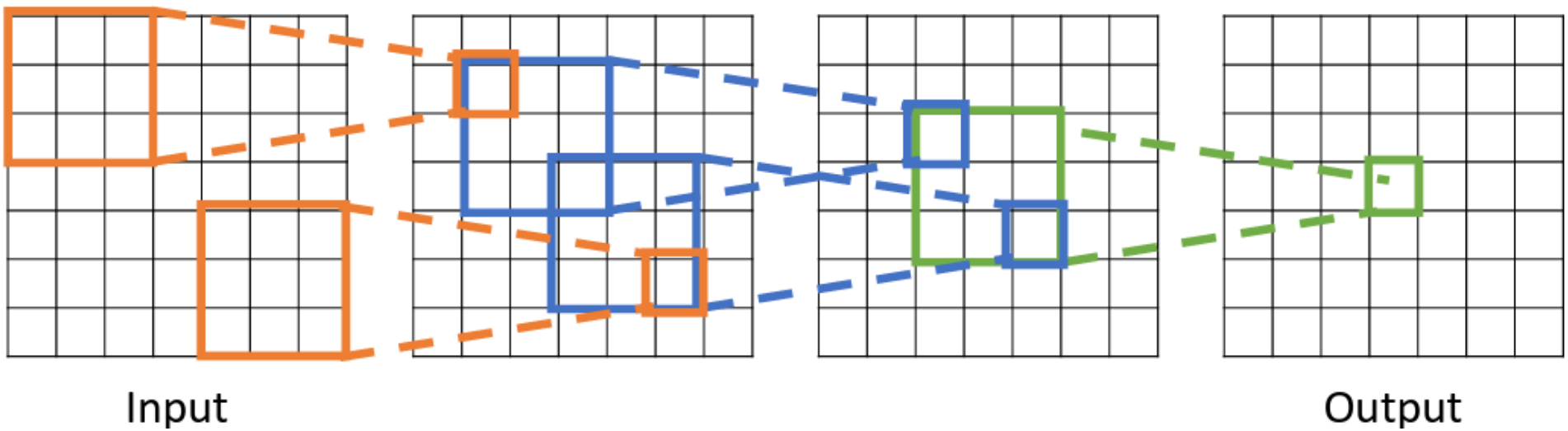
CNNs-Convolution Layer

➢ A Closer Look at Receptive Fields

Input size: C_{in} x W x W **Kernel size: K x K**

In order for each output neuron to "see" the whole image, *L*=?

Why do we care about if each output neuron "sees" the whole input image?



Problem: For large input images, we need many layers for each output neuron to "see" the whole input image!

Solution: Downsample inside CNN!

CNNs-Convolution Layer

➢ Solution 1 for Downsampling: Strided Convolution

The step size of the filter's movement

Input size: C_{in} x W x W

Kernel size: K x K

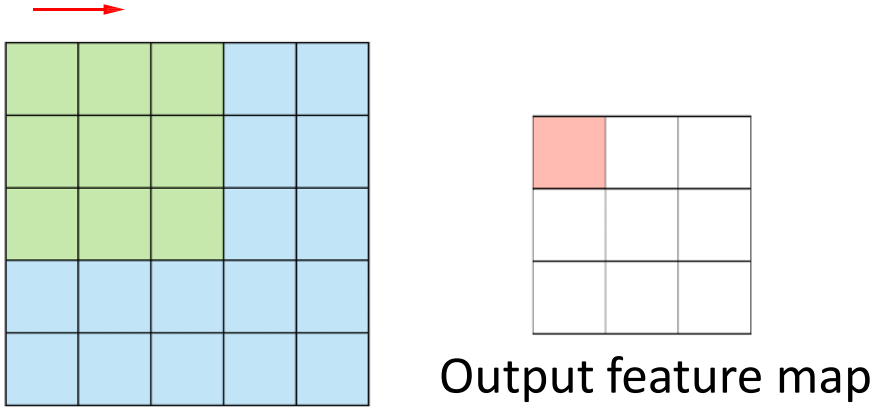
Zero-Padding size: P

Stride size: S

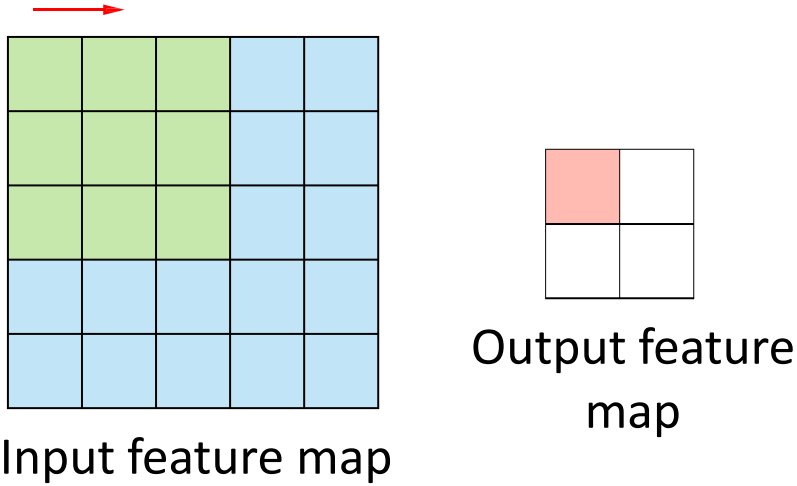
Output size (Spatially): ?

(Only show the spatial dimensions for visualization)

Stride S=1



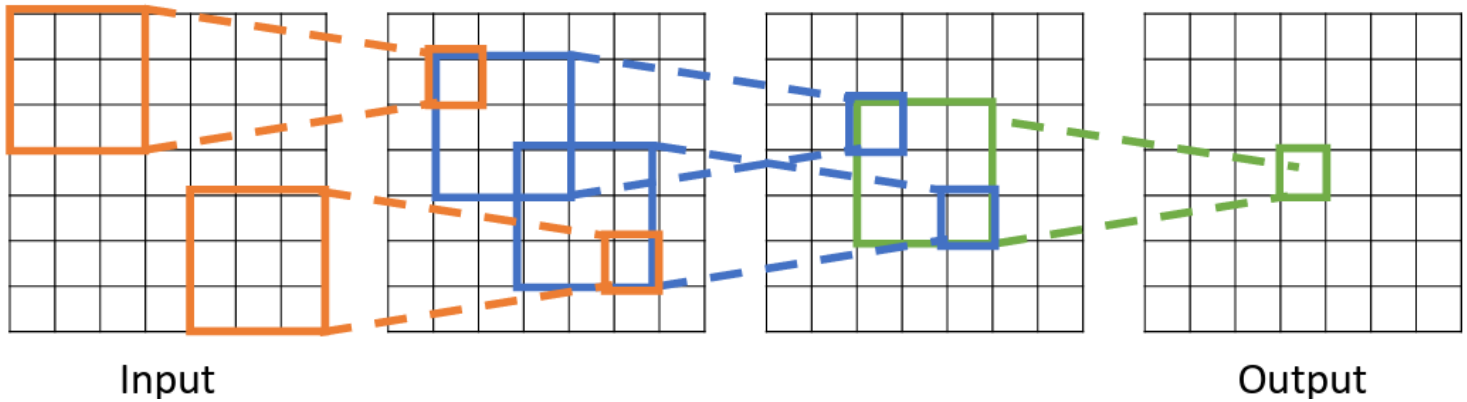
Stride S=2



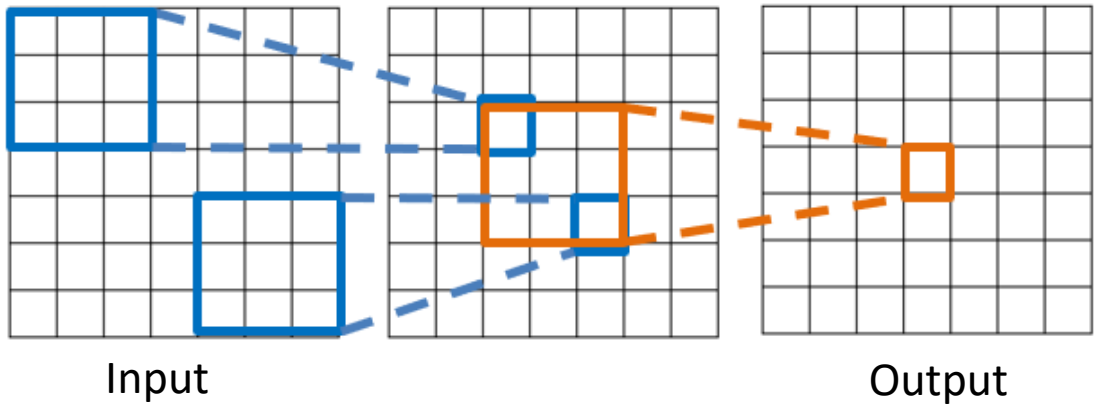
CNNs-Convolution Layer

➢ Solution 1 for Downsampling: Strided Convolution

Stride S=1, need **3** convolution layers to "see" the whole input image



Stride S=2, need **2** convolution layers to "see" the whole input image



Reduced number of layers needed to "see" the whole input image

CNNs-Convolution Layer

➢ Solution 1 for Downsampling: Strided Convolution

Each successive convolution adds **?** to the receptive field size

Stride: S, Kernel Size: K, Conv layer#: L

CNNs-Convolution Layer

➢ Solution 1 for Downsampling: Strided Convolution

Each successive convolution adds **?** to the receptive field size

Stride: S, Kernel Size: K, Conv layer#: L

1 layer: $(RF_1-K)/S + 1 = 1 \longrightarrow RF_1 = K$

2 layers: $(RF_2-K)/S + 1 = RF_1 \longrightarrow RF_2 = (K-1)S + K$

3 layers: $(RF_3-K)/S + 1 = RF_2 \longrightarrow RF_3 = S((K-1)S+K-1)+K=(K-1)S^2+(K-1)S+K$

⋮

L layers: $(RF_L-K)/S + 1 = RF_{L-1} \longrightarrow RF_L =(K-1)S^{L-1}+ RF_{L-1}$

CNNs-Convolution Layer

➢ Solution 1 for Downsampling: Strided Convolution

Practice Question 1

Input volume: 3x32x32

10 filters, each has a kernel size of 3x3

Stride 3, Zero-padding 2

Output volume size: ?

Number of learnable parameters (including biases): ?

CNNs-Convolution Layer

➢ Solution 1 for Downsampling: Strided Convolution

Practice Question 1

Input volume: 3x32x32

10 filters, each has a kernel size of 3x3

Stride 3, Zero-padding 2

Output volume size: ?

(W+2P-K)/S +1=(32+2x2-3)/3+1=12 spatially, so 10 x12x12

Number of learnable parameters (including biases): ?

10x(3x3x3+1)=280

CNNs-Convolution Layer

➢ Solution 1 for Downsampling: Strided Convolution

Practice Question 2

Input volume: 2x2x2

Input channel 1:

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

Input channel 2:

$$\begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}$$

one 3x3 filter with stride 1, pad 1

Filter channel 1:

$$\begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$$

Filter channel 2:

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

Output?

CNNs-Convolution Layer

➢ Solution 1 for Downsampling: Strided Convolution

Practice Question 2

Input volume: 2x2x2, one 3x3 filter with stride 1, pad 1

Output?

$$\begin{bmatrix} 9 & 7 \\ 7 & 9 \end{bmatrix}$$

Input channel 1:

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

Input channel 2:

$$\begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}$$

Filter channel 1:

$$\begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$$

Filter channel 2:

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

Solution:

Step 1: Zero-padding

Input channel 1:

$$\begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 2 & 0 \\ 0 & 3 & 4 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Input channel 2:

$$\begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 2 & 0 \\ 0 & 2 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Step 2: Convolution

$$O(1,1)=\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 2 \\ 0 & 3 & 4 \end{bmatrix}\cdot\begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}+\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 2 \\ 0 & 2 & 1 \end{bmatrix}\cdot\begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}=5+4=9$$

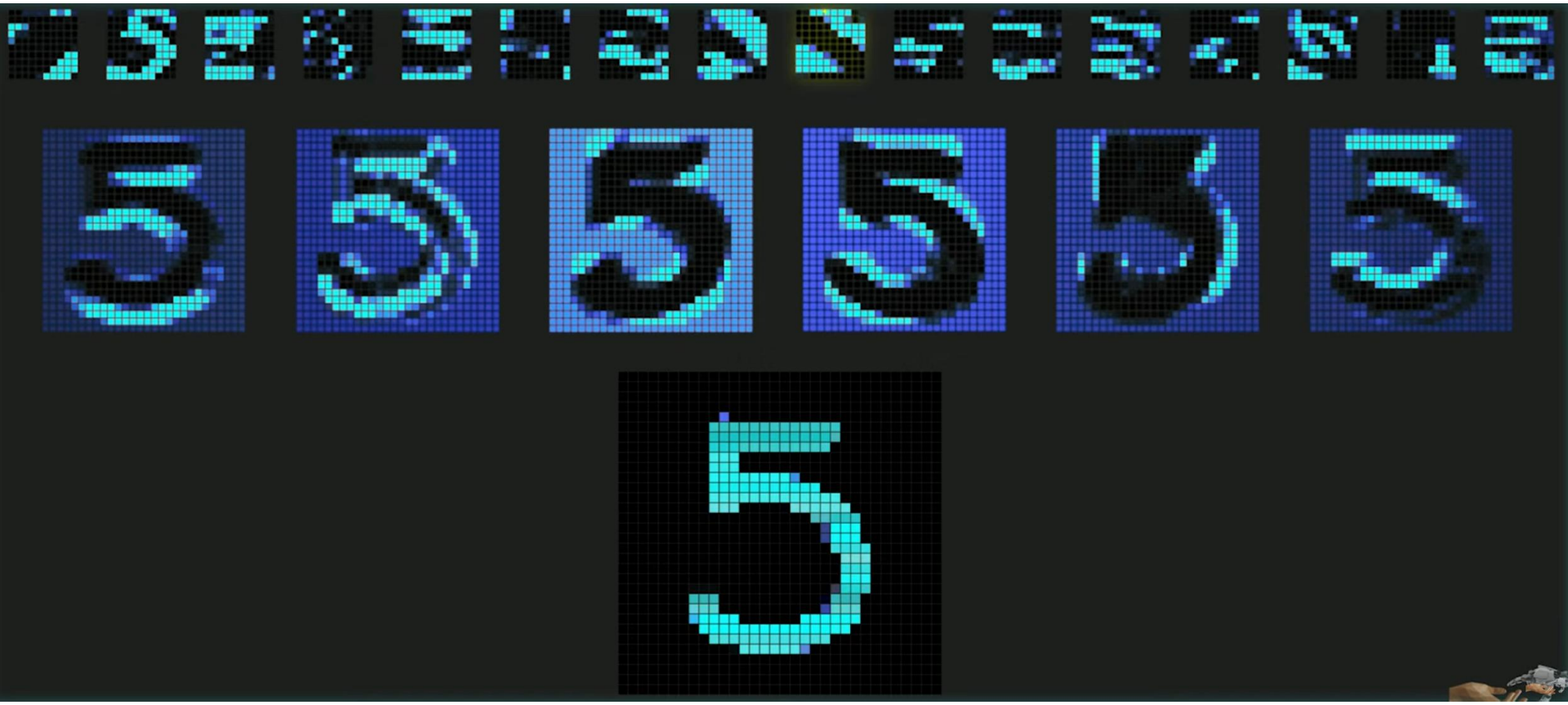
$$O(1,2)=\begin{bmatrix} 0 & 0 & 0 \\ 1 & 2 & 0 \\ 3 & 4 & 0 \end{bmatrix}\cdot\begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}+\begin{bmatrix} 0 & 0 & 0 \\ 1 & 2 & 0 \\ 2 & 1 & 0 \end{bmatrix}\cdot\begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}=5+2=7$$

$$O(2,1)=\begin{bmatrix} 0 & 1 & 2 \\ 0 & 3 & 4 \\ 0 & 0 & 0 \end{bmatrix}\cdot\begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}+\begin{bmatrix} 0 & 1 & 2 \\ 0 & 2 & 1 \\ 0 & 0 & 0 \end{bmatrix}\cdot\begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}=5+2=7$$

$$O(2,2)=\begin{bmatrix} 1 & 2 & 0 \\ 3 & 4 & 0 \\ 0 & 0 & 0 \end{bmatrix}\cdot\begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}+\begin{bmatrix} 1 & 2 & 0 \\ 2 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}\cdot\begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}=5+4=9$$

CNNs-Convolution Layer

➢ Visualization of Activation Maps

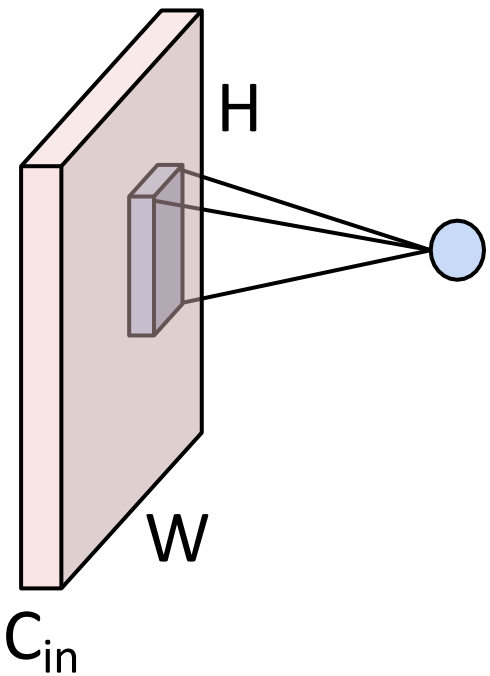


CNNs-Convolution Layer

➢ Other Types of Convolution

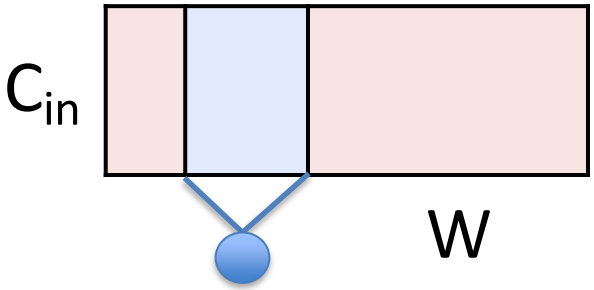
So far: 2D Convolution

Input: C_{in} x H x W
Weight Matrix: C_{out} x C_{in} x K_H x K_W
Output: C_{out} x H' x W'



- **1D Convolution**

Input: C_{in} x W
Weight Matrix: C_{out} x C_{in} x K_W
Output: C_{out} x W'



e.g., one input: 2x5, one filter: 2x3, stride: 2, padding: 1

Input channel 1: [1,2,3,4,5]
Input channel 2:[0,1,2,1,0]

Filter channel 1: [1,0,1]
input channel 2:[-1,1,-1]

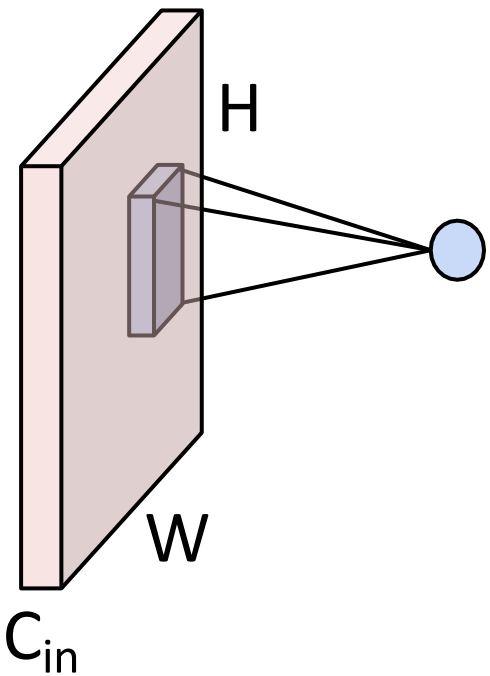
Output size? Output?

CNNs-Convolution Layer

➢ Other Types of Convolution

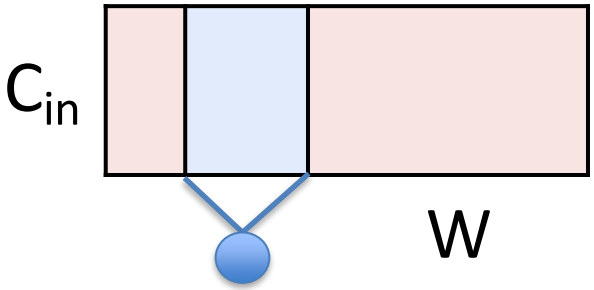
So far: 2D Convolution

Input: C_{in} x H x W
Weight Matrix: C_{out} x C_{in} x K_H x K_W
Output: C_{out} x H' x W'



- **1D Convolution**

Input: C_{in} x W
Weight Matrix: C_{out} x C_{in} x K_W
Output: C_{out} x W'



e.g., one input: 2x5, one filter: 2x3, stride: 2, padding: 1

Input channel 1: [1,2,3,4,5]
Input channel 2:[0,1,2,1,0]

Filter channel 1: [1,0,1]
input channel 2:[-1,1,-1]

Solution:

Step 1: Zero-padding

Input channel 1: [0,1,2,3,4,5,0]
Input channel 2:[0,0,1,2,1,0,0]

Step 2: Convolution

O(1)=[0,1,2]·[1,0,1]+ [0,0,1]·[-1,1,-1]=1
O(2)=[2,3,4]·[1,0,1]+ [1,2,1]·[-1,1,-1]=6
O(3)=[4,5,0]·[1,0,1]+ [1,0,0]·[-1,1,-1]=3

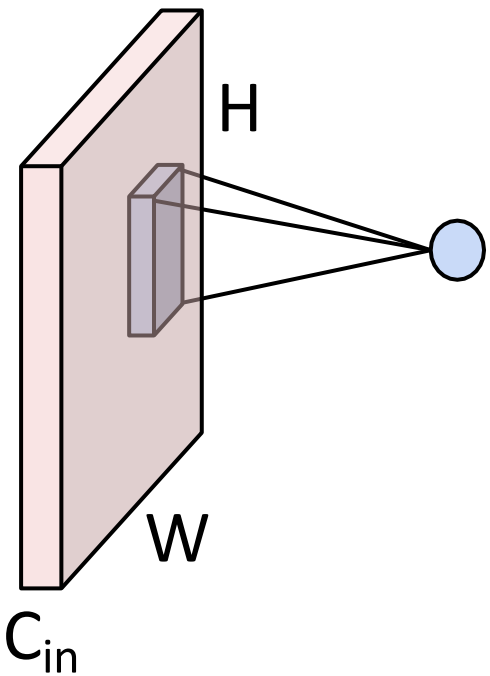
Output=[1,6,3]

CNNs-Convolution Layer

➢ Other Types of Convolution

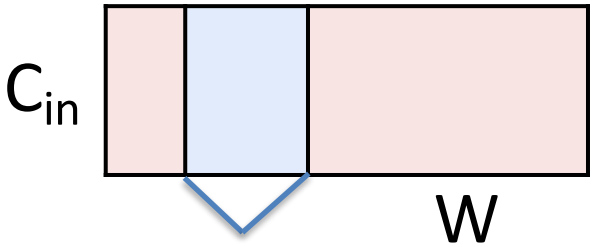
So far: 2D Convolution

Input: C_{in} x H x W
Weight Matrix: C_{out} x C_{in} x K_H x K_W
Output: C_{out} x H' x W'

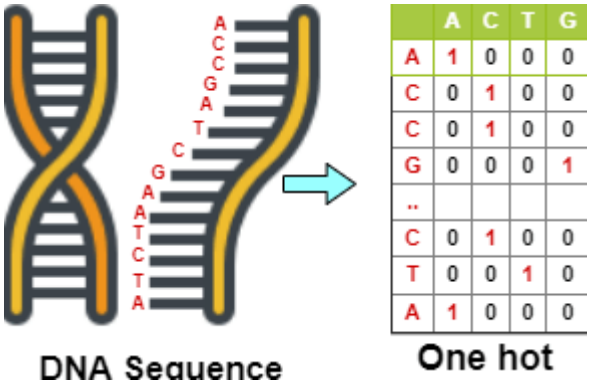


- **1D Convolution**

Input: C_{in} x W
Weight Matrix: C_{out} x C_{in} x K_W
Output: C_{out} x W'



Extracting local patterns from ***sequential*** or ***time-dependent*** data



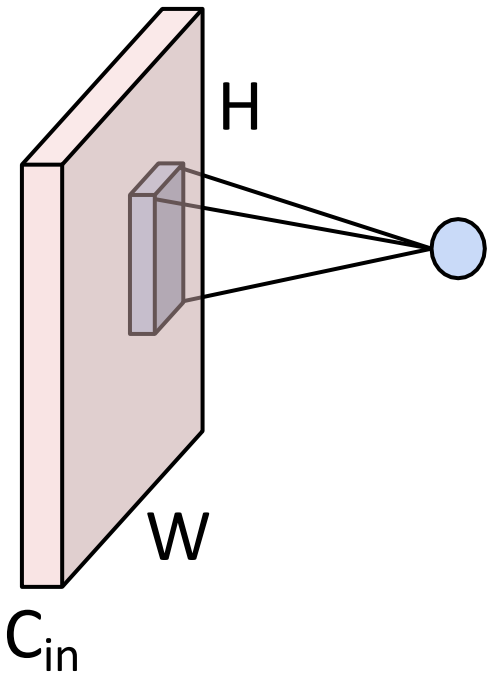
	A	C	T	G
A	1	0	0	0
C	0	1	0	0
C	0	1	0	0
G	0	0	0	1
..				
C	0	1	0	0
T	0	0	1	0
A	1	0	0	0

CNNs-Convolution Layer

➢ Other Types of Convolution

So far: 2D Convolution

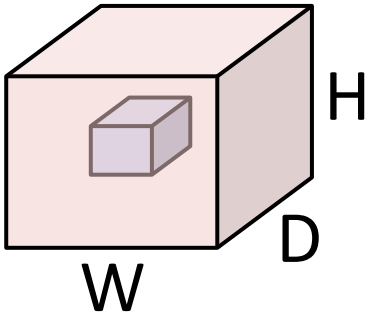
Input: $C_{in} \times H \times W$
Weight Matrix: $C_{out} \times C_{in} \times K_H \times K_W$
Output: $C_{out} \times H' \times W'$



- **3D Convolution**

Input: $C_{in} \times D \times H \times W$
Weight Matrix: $C_{out} \times C_{in} \times K_D \times K_H \times K_W$
Output: $C_{out} \times D' \times H' \times W'$

C_{in}-dim vector at each point in the volume



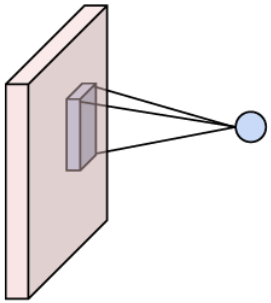
Track how a spatial pattern changes over depth or time (spatiotemporal data)

e.g.,

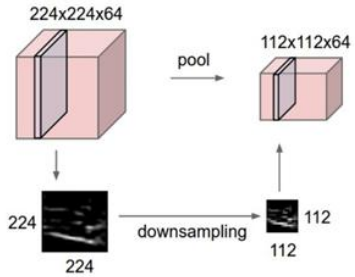
CNNs

➢ Components of A Typical CNN

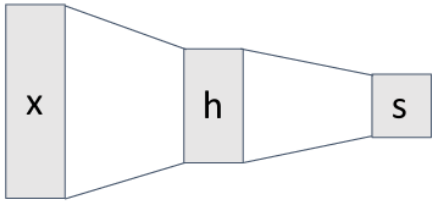
Convolution Layers



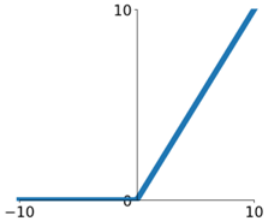
Pooling Layers



Fully-Connected Layers



Activation Function



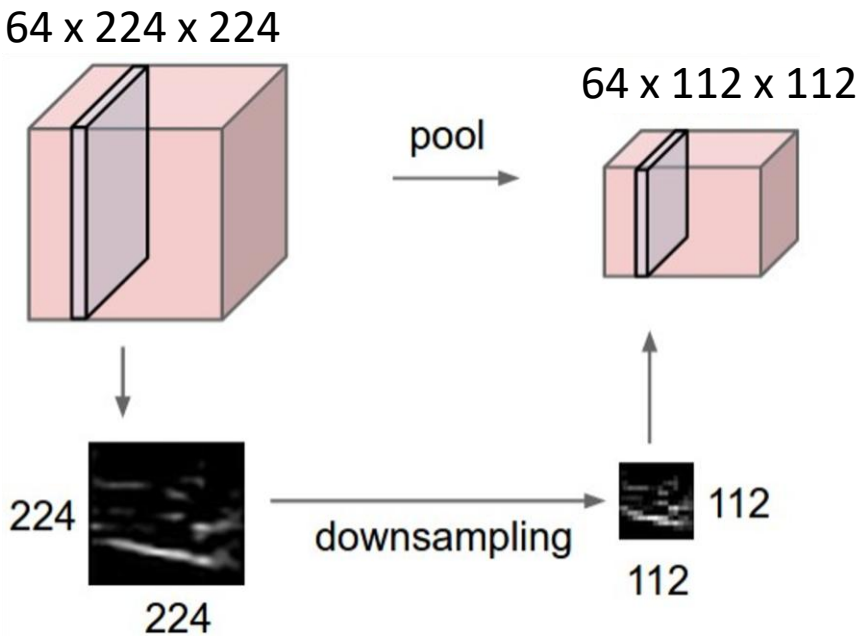
Normalization

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

CNNs-Pooling Layer

➢ Solution 2 for Downsampling

Downsample each activation map independently



Hyperparameters:
Kernel Size *K*
Stride *S*
Pooling function

Single Channel Slice

10	13	65	18
20	21	32	21
16	17	21	3
8	23	25	43

2x2 pooling, stride 2

Max pooling

21	65
23	43

Average pooling

16	34
16	23

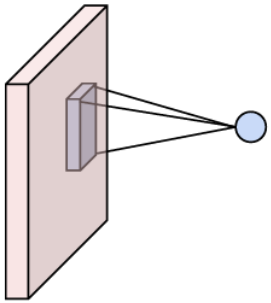
Global Max
65

Global Average
22.25

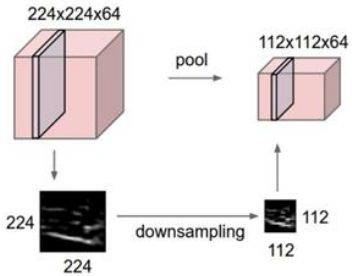
CNNs

➢ Components of A Typical CNN

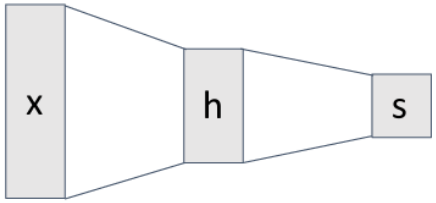
Convolution Layers



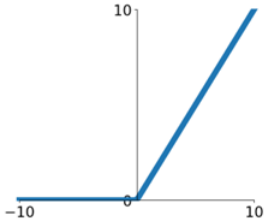
Pooling Layers



Fully-Connected Layers



Activation Function



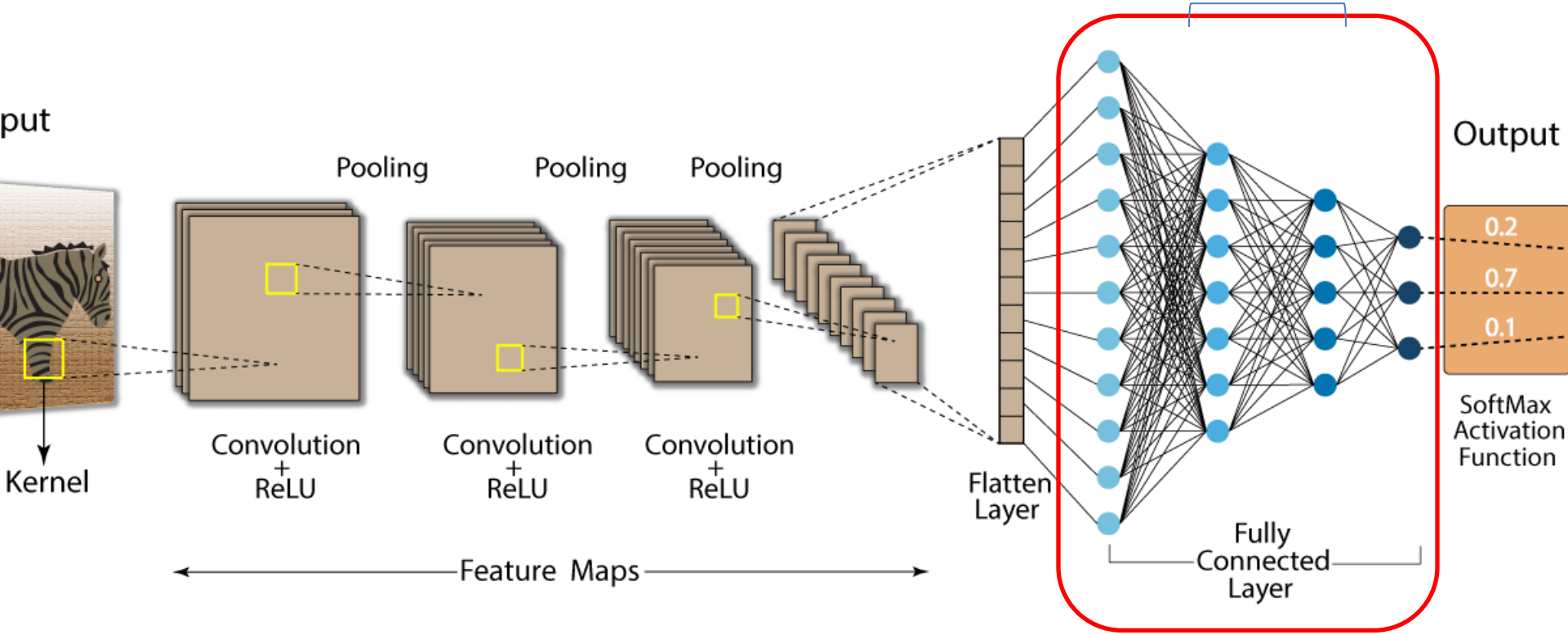
Normalization

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

CNNs-Fully Connected Layer

- **Every neuron in a layer is connected to all the neurons in the previous layer**

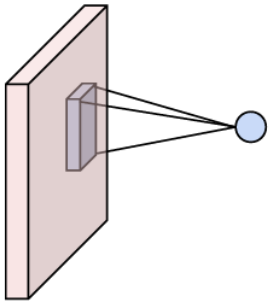
Activation Function: ReLU



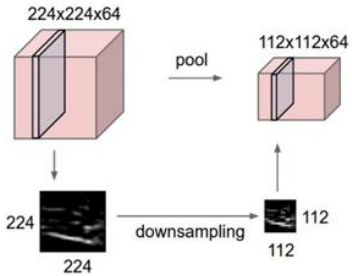
CNNs

➢ Components of A Typical CNN

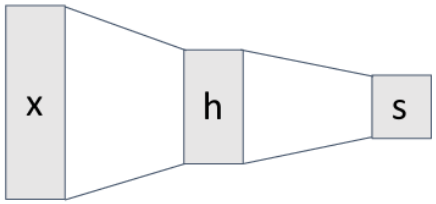
Convolution Layers



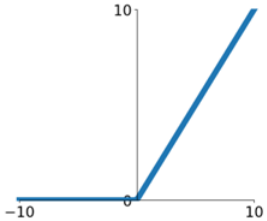
Pooling Layers



Fully-Connected Layers



Activation Function



Normalization

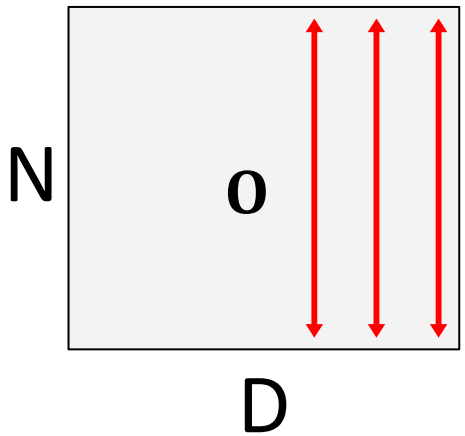
$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

CNNs-Normalization

➢ Batch Normalization (BN)

Idea: "Normalize" the outputs of a layer so they have **zero mean** and **unit variance** (over the batch) Why?

Input to BN: $\mathbf{o} \in \mathbb{R}^{N \times D}$



$$\mu_j = \frac{1}{N}\sum_{i=1}^{N} o_{i,j}$$

Per-feature mean

$\boldsymbol{\mu}$: D-dim vector

$$\sigma_j^2 = \frac{1}{N}\sum_{i=1}^{N} (o_{i,j} - \mu_j)^2$$

Per-feature variance

$\boldsymbol{\sigma}$: D-dim vector

$$\hat{o}_{i,j} = \frac{o_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \boxed{\varepsilon}}}$$

Numerical stability

$\hat{\mathbf{o}}$:Normalized $\mathbf{o}$, $\in \mathbb{R}^{N \times D}$

Ioffe and Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift", ICML 2015

CNNs-Normalization

➢ Batch Normalization (BN)

Why BN? Help reduce "**internal covariate shift**"

The distribution of inputs to a layer changes during training, because the parameters of the previous layers change.

e.g, Consider a 3-layer network

	Layer 1	→	Layer 2	→	Layer 3
Forward Pass:	$\mathbf{W}_{old}^{(1)}$		$\mathbf{W}_{old}^{(2)}$		$\mathbf{W}_{old}^{(3)}$
output	$\mathbf{O}_{old}^{(1)}$		$\mathbf{O}_{old}^{(2)}$		$\mathbf{O}_{old}^{(3)}$
Backward Pass:					
error	$\mathbf{E}^{(1)} \leftarrow f(\mathbf{E}^{(2)}, \mathbf{W}_{old}^{(2)}, \mathbf{O}_{old}^{(1)})$		$\mathbf{E}^{(2)} \leftarrow f(\mathbf{E}^{(3)}, \mathbf{W}_{old}^{(3)}, \mathbf{O}_{old}^{(2)})$		$\mathbf{E}^{(3)} \leftarrow f(\mathbf{O}_{old}^{(3)}, \mathbf{Y})$
update	$\mathbf{W}_{new}^{(1)} \leftarrow f(\mathbf{W}_{old}^{(1)}, \mathbf{E}^{(1)}, \mathbf{X})$		$\mathbf{W}_{new}^{(2)} \leftarrow f(\mathbf{W}_{old}^{(2)}, \mathbf{E}^{(2)}, \mathbf{O}_{old}^{(1)})$		$\mathbf{W}_{new}^{(3)} \leftarrow f(\mathbf{W}_{old}^{(3)}, \mathbf{E}^{(3)}, \mathbf{O}_{old}^{(2)})$
Forward Pass:	$\mathbf{O}_{NEW}^{(1)}$		$\mathbf{O}_{NEW}^{(2)}$		$\mathbf{O}_{NEW}^{(3)}$

Ioffe and Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift", ICML 2015

CNNs-Normalization

➢ Batch Normalization (BN)

Idea: "Normalize" the outputs of a layer so they have **zero mean** and **unit variance** (over the batch)

Problem?

Ioffe and Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift", ICML 2015

CNNs-Normalization

➢ Batch Normalization (BN)

Idea: "Normalize" the outputs of a layer so they have **zero mean** and **unit variance** (over the batch)

May distort the representation the network actually wants

(Different layers, different features may have different range for optimal learning)

Solution: Add learnable scale γ and shift β!

Ioffe and Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift", ICML 2015

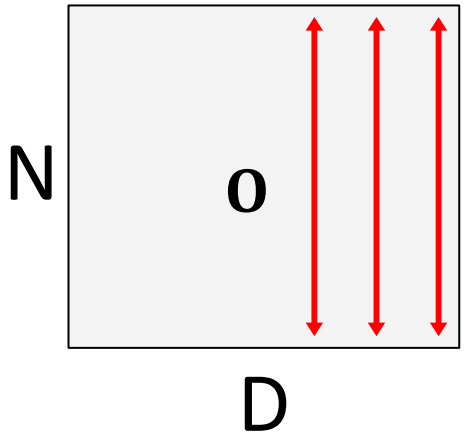
CNNs-Normalization

➢ Batch Normalization (BN)

Idea: "Normalize" the outputs of a layer so they have zero mean and unit variance (over the batch)

Input to BN: $\mathbf{o} \in \mathbb{R}^{N \times D}$

Problem: Estimates depend on minibatch; can't do this at test-time!



$$\mu_j = \frac{1}{N}\sum_{i=1}^{N} o_{i,j}$$

$$\sigma_j^2 = \frac{1}{N}\sum_{i=1}^{N} (o_{i,j} - \mu_j)^2$$

$$\hat{o}_{i,j} = \frac{o_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

Per-feature mean

$\boldsymbol{\mu}$: D-dim vector

Per-feature variance

$\boldsymbol{\sigma}$: D-dim vector

$\hat{\mathbf{o}}$:Normalized $\mathbf{o}$, $\in \mathbb{R}^{N \times D}$

learnable scale $\boldsymbol{\gamma}$ and shift $\boldsymbol{\beta}$: $\boldsymbol{\gamma}, \boldsymbol{\beta} \in \mathbb{R}^D$

$$\tilde{o}_{i,j} = \gamma_j \hat{o}_{i,j} + \beta_j$$

$\tilde{\mathbf{o}} \in \mathbb{R}^{N \times D}$

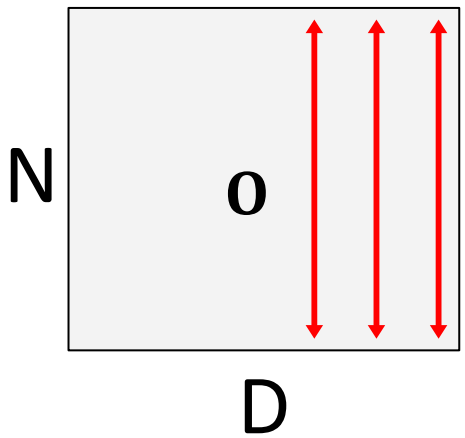
Ioffe and Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift", ICML 2015

CNNs-Normalization

➢ Batch Normalization (BN)

TEST TIME:

Input to BN: $\mathbf{o} \in \mathbb{R}^{N \times D}$



$$\mu_j = Aggregated\ mean\ during\ training$$

$$\sigma_j^2 = Aggregated\ variance\ during\ training$$

$$\hat{o}_{i,j} = \frac{o_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

$$\tilde{o}_{i,j} = \gamma_j^* \hat{o}_{i,j} + \beta_j^*$$

Ioffe and Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift", ICML 2015

CNNs-Normalization

➢ Batch Normalization (BN)

Example: Batch size (N)=3, D (No. features)=2, $\mathbf{O} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}$

Scale $\boldsymbol{\gamma}$=[2,3], Shift $\boldsymbol{\beta} = [1, 2]$,

What is the output after batch normalization?

Ioffe and Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift", ICML 2015

CNNs-Normalization

➢ Batch Normalization (BN)

Example: Batch size (N)=3, D (No. features)=2, $\mathbf{O} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}$

Scale $\boldsymbol{\gamma}$=[2,3], Shift $\boldsymbol{\beta} = [1, 2]$,

What is the output after batch normalization?

Step 1: Compute per-feature mean and standard deviation

$\boldsymbol{\mu}$=[3,4], $\boldsymbol{\sigma}$=[1.63, 1.63]

Step 2: Compute $\widehat{\mathbf{O}}$ (Normalized $\mathbf{O}$)

$$\widehat{\mathbf{O}} = \begin{bmatrix} \frac{1-3}{1.63} & \frac{2-4}{1.63} \\ \frac{3-3}{1.63} & \frac{4-4}{1.63} \\ \frac{5-3}{1.63} & \frac{6-4}{1.63} \end{bmatrix} = \begin{bmatrix} -1.23 & -1.23 \\ 0 & 0 \\ 1.23 & 1.23 \end{bmatrix}$$

Step 3: Scaling & Shifting

$$\widetilde{\mathbf{O}} = \begin{bmatrix} -1.23 \times 2 + 1 & -1.23 \times 3 + 2 \\ 0 \times 2 + 1 & 0 \times 3 + 2 \\ 1.23 \times 2 + 1 & 1.23 \times 3 + 2 \end{bmatrix}$$

$$= \begin{bmatrix} -1.46 & -1.69 \\ 1 & 2 \\ 3.46 & 5.69 \end{bmatrix}$$

Ioffe and Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift", ICML 2015

CNNs-Normalization

➢ Batch Normalization (BN)

✓ **For FC layers**

$$\mathbf{O}: N \times D$$

Normalize

$$\boldsymbol{\mu}, \boldsymbol{\sigma}: 1 \times D$$

$$\boldsymbol{\gamma}, \boldsymbol{\beta}: 1 \times D$$

$$\tilde{\mathbf{o}} = \boldsymbol{\gamma} * \frac{(\mathbf{o} - \boldsymbol{\mu})}{\boldsymbol{\sigma} + \varepsilon} + \boldsymbol{\beta}$$

✓ **For Conv layers**
(2D Conv)

$$\mathbf{O}: N \times C \times H \times W$$

Normalize

$$\boldsymbol{\mu}, \boldsymbol{\sigma}: 1 \times C \times 1 \times 1$$

$$\boldsymbol{\gamma}, \boldsymbol{\beta}: 1 \times C \times 1 \times 1$$

$$\tilde{\mathbf{o}} = \boldsymbol{\gamma} * \frac{(\mathbf{o} - \boldsymbol{\mu})}{\boldsymbol{\sigma} + \varepsilon} + \boldsymbol{\beta}$$

Ioffe and Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift", ICML 2015

CNNs-Normalization

➢ Batch Normalization (BN)

Example: Batch size (N) = 2,
Input to BN = 2 x 2 x 2,

Image 1: Channel 1: $\begin{bmatrix} 2 & 2 \\ 2 & 2 \end{bmatrix}$

Channel 2: $\begin{bmatrix} 10 & 10 \\ 10 & 10 \end{bmatrix}$

Image 2: Channel 1: $\begin{bmatrix} 6 & 6 \\ 6 & 6 \end{bmatrix}$

Channel 2: $\begin{bmatrix} 20 & 20 \\ 20 & 20 \end{bmatrix}$

$\boldsymbol{\gamma} = [2, 10]$, $\boldsymbol{\beta} = [5, 0]$

What is the output after batch normalization?

✓ **For Conv layers**
(2D Conv)

$$\mathbf{O}: N \times C \times H \times W$$

Normalize

$$\boldsymbol{\mu}, \boldsymbol{\sigma} : 1 \times C \times 1 \times 1$$

$$\boldsymbol{\gamma}, \boldsymbol{\beta} : 1 \times C \times 1 \times 1$$

$$\tilde{\mathbf{o}} = \boldsymbol{\gamma} * \frac{(\mathbf{o} - \boldsymbol{\mu})}{\boldsymbol{\sigma} + \varepsilon} + \boldsymbol{\beta}$$

Ioffe and Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift", ICML 2015

CNNs-Normalization

➢ Batch Normalization (BN)

Example: Batch size (N) = 2,
Input to BN = 2 x 2 x 2,

Image 1: Channel 1: $\begin{bmatrix} 2 & 2 \\ 2 & 2 \end{bmatrix}$

Channel 2: $\begin{bmatrix} 10 & 10 \\ 10 & 10 \end{bmatrix}$

Image 2: Channel 1: $\begin{bmatrix} 6 & 6 \\ 6 & 6 \end{bmatrix}$

Channel 2: $\begin{bmatrix} 20 & 20 \\ 20 & 20 \end{bmatrix}$

$\boldsymbol{\gamma} = [2, 10]$, $\boldsymbol{\beta} = [5, 0]$

What is the output after batch normalization?

Step 1: Compute per-feature mean and standard deviation

$$\mu_1 = \frac{(2+6)\times 4}{8} = 4$$

$$\mu_2 = \frac{(10+20)\times 4}{8} = 15$$

$$\sigma_1 = 2, \sigma_2 = 5$$

Step 2: Normalize

Image 1: Channel 1: $\begin{bmatrix} -1 & -1 \\ -1 & -1 \end{bmatrix}$

Channel 2: $\begin{bmatrix} -1 & -1 \\ -1 & -1 \end{bmatrix}$

Image 2: Channel 1: $\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$

Channel 2: $\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$

Ioffe and Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift", ICML 2015

CNNs-Normalization

➢ Batch Normalization (BN)

Example: Batch size (N) = 2,
Input to BN = 2 x 2 x 2,

Image 1: Channel 1: $\begin{bmatrix} 2 & 2 \\ 2 & 2 \end{bmatrix}$

Channel 2: $\begin{bmatrix} 10 & 10 \\ 10 & 10 \end{bmatrix}$

Image 2: Channel 1: $\begin{bmatrix} 6 & 6 \\ 6 & 6 \end{bmatrix}$

Channel 2: $\begin{bmatrix} 20 & 20 \\ 20 & 20 \end{bmatrix}$

$\boldsymbol{\gamma} = [2, 10]$, $\boldsymbol{\beta} = [5, 0]$

What is the output after batch normalization?

Step 2: Normalize

Image 1: Channel 1: $\begin{bmatrix} -1 & -1 \\ -1 & -1 \end{bmatrix}$

Channel 2: $\begin{bmatrix} -1 & -1 \\ -1 & -1 \end{bmatrix}$

Image 2: Channel 1: $\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$

Channel 2: $\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$

Step 3: Scale & Shift:

Image 1: Channel 1: $\begin{bmatrix} 3 & 3 \\ 3 & 3 \end{bmatrix}$

Channel 2: $\begin{bmatrix} -10 & -10 \\ -10 & -10 \end{bmatrix}$

Image 2: Channel 1: $\begin{bmatrix} 7 & 7 \\ 7 & 7 \end{bmatrix}$

Channel 2: $\begin{bmatrix} 10 & 10 \\ 10 & 10 \end{bmatrix}$

Ioffe and Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift", ICML 2015

CNNs-Normalization

➢ Batch Normalization (BN)

✓ **For 1D Conv layers**

$$\mathbf{O}: N \times C \times W$$

Normalize

$$\boldsymbol{\mu}, \boldsymbol{\sigma} : 1 \times C \times 1$$

$$\boldsymbol{\gamma}, \boldsymbol{\beta} : 1 \times C \times 1$$

$$\tilde{\mathbf{o}} = \boldsymbol{\gamma} * \frac{(\mathbf{o} - \boldsymbol{\mu})}{\boldsymbol{\sigma} + \varepsilon} + \boldsymbol{\beta}$$

✓ **For 3D Conv layers**

$$\mathbf{O}: N \times C \times D \times H \times W$$

Normalize

$$\boldsymbol{\mu}, \boldsymbol{\sigma} : 1 \times C \times 1 \times 1 \times 1$$

$$\boldsymbol{\gamma}, \boldsymbol{\beta} : 1 \times C \times 1 \times 1 \times 1$$

$$\tilde{\mathbf{o}} = \boldsymbol{\gamma} * \frac{(\mathbf{o} - \boldsymbol{\mu})}{\boldsymbol{\sigma} + \varepsilon} + \boldsymbol{\beta}$$

Ioffe and Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift”, ICML 2015

THE END